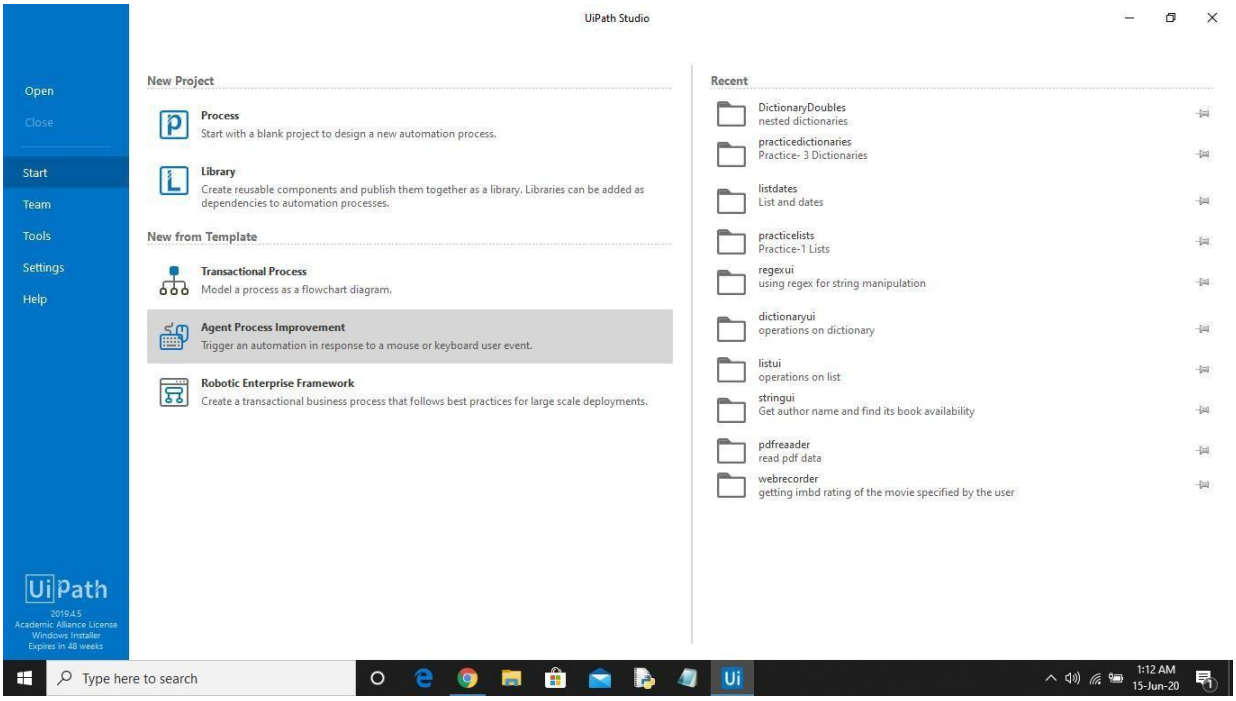
Practical 1: Student Attendance Eligibility Using Sequence and If Activity

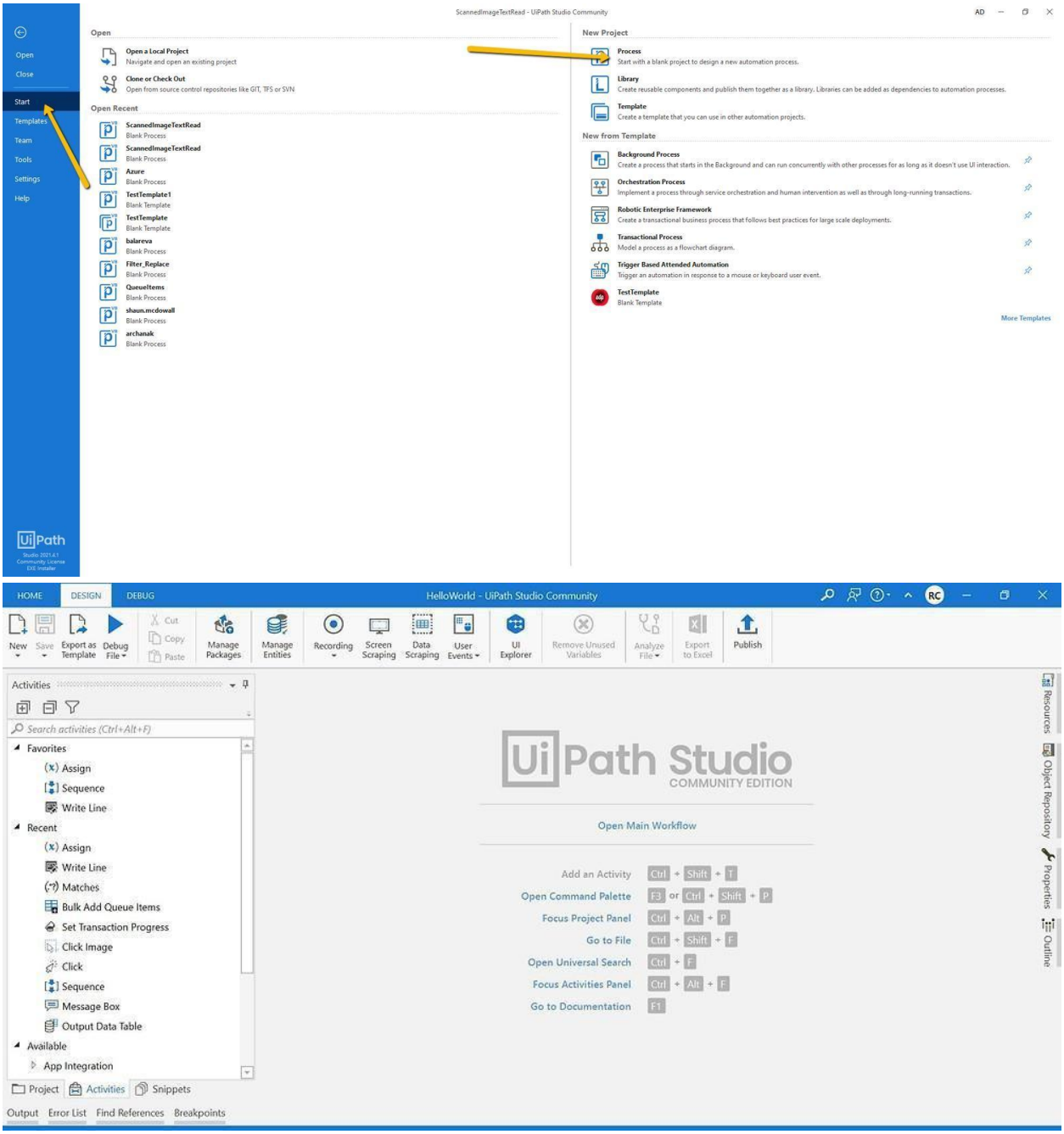
Aim

Develop a UiPath robot that checks whether a student is eligible for an examination based on attendance.

Eligibility Rule: Attendance ≥ 75%

Step 1: Open UiPath Studio





4

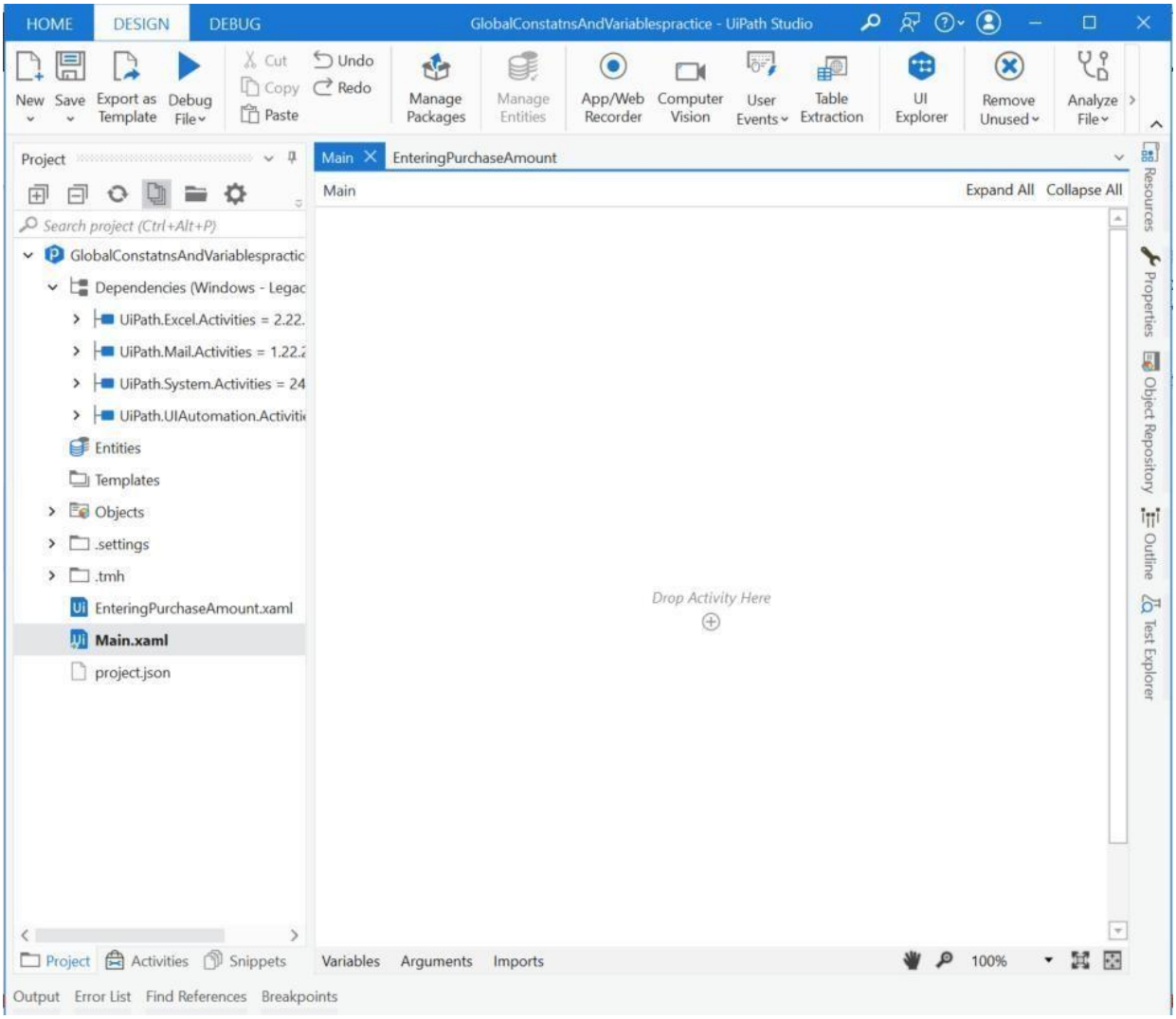
Procedure

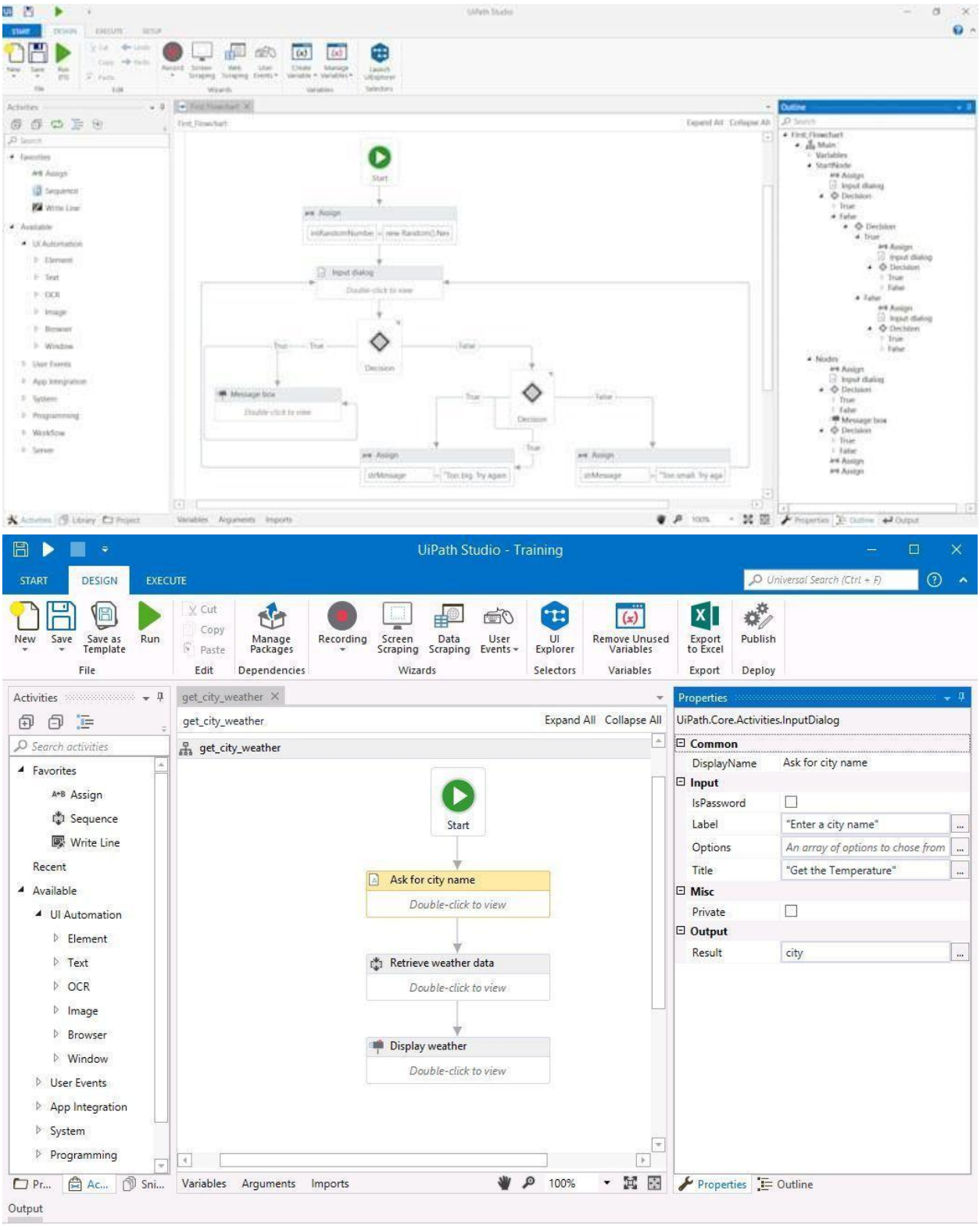
1. Open **UiPath Studio Desktop**.
2. Click **New Project**.
3. Select **Process**.
4. Enter the project name:

AttendanceEligibility

5. Click **Create**.

Step 2: Open Main.xaml



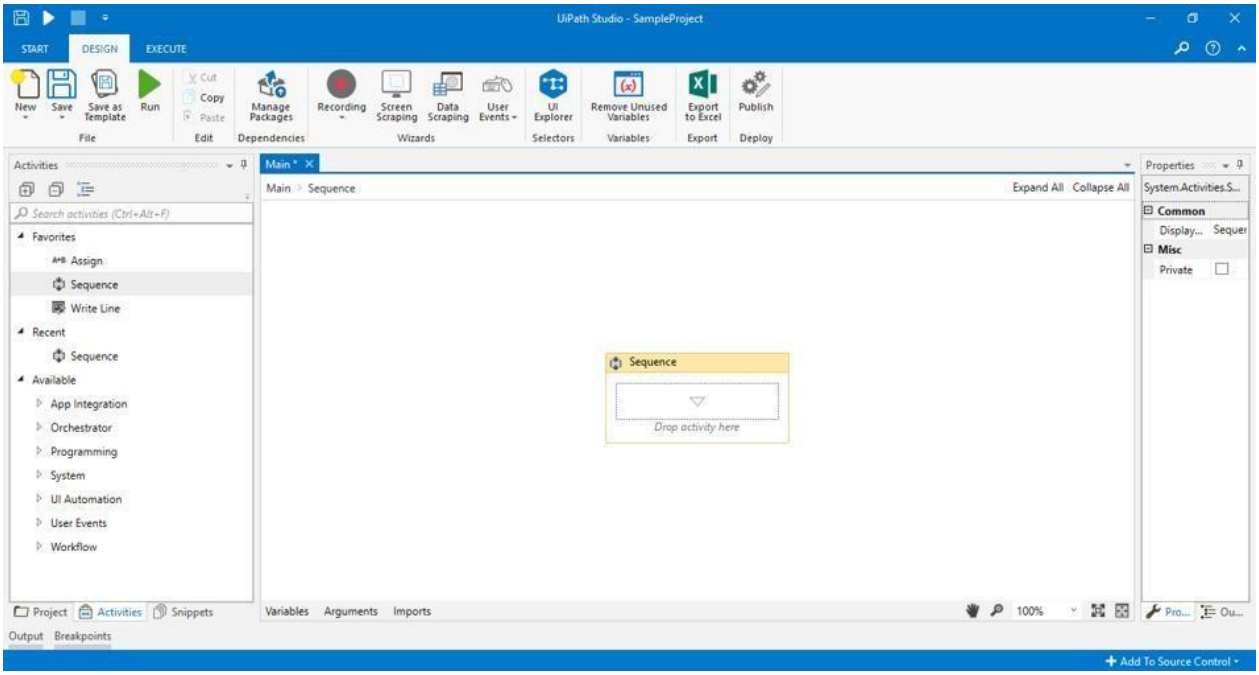


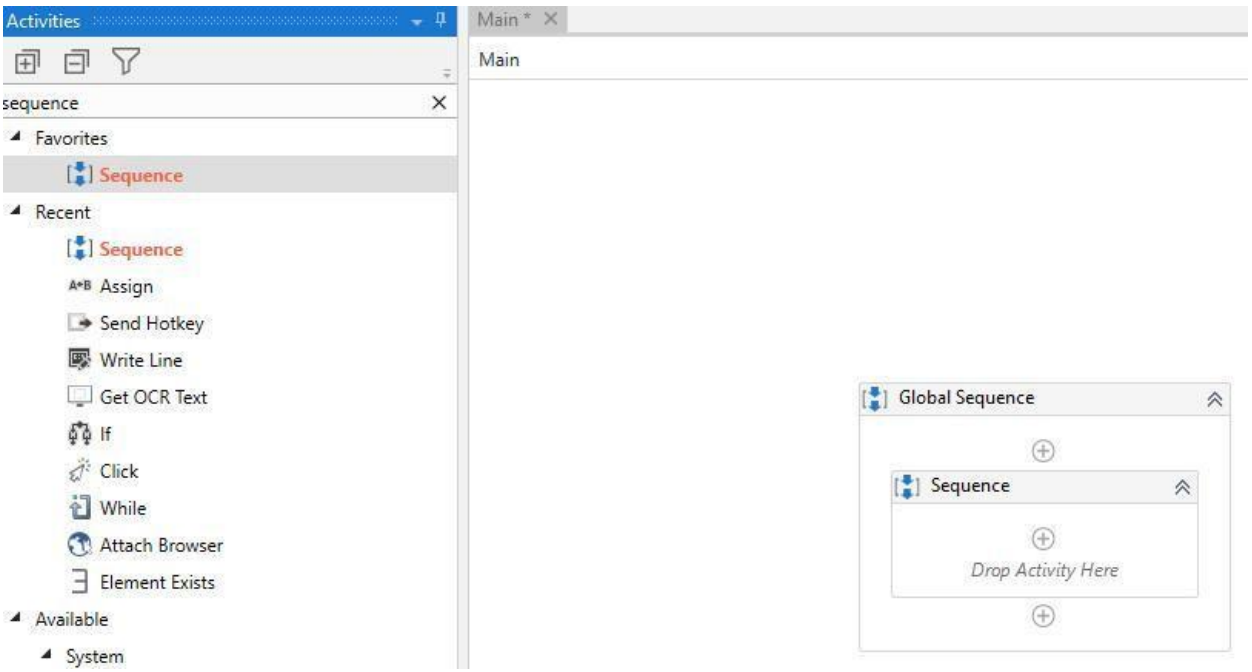
4

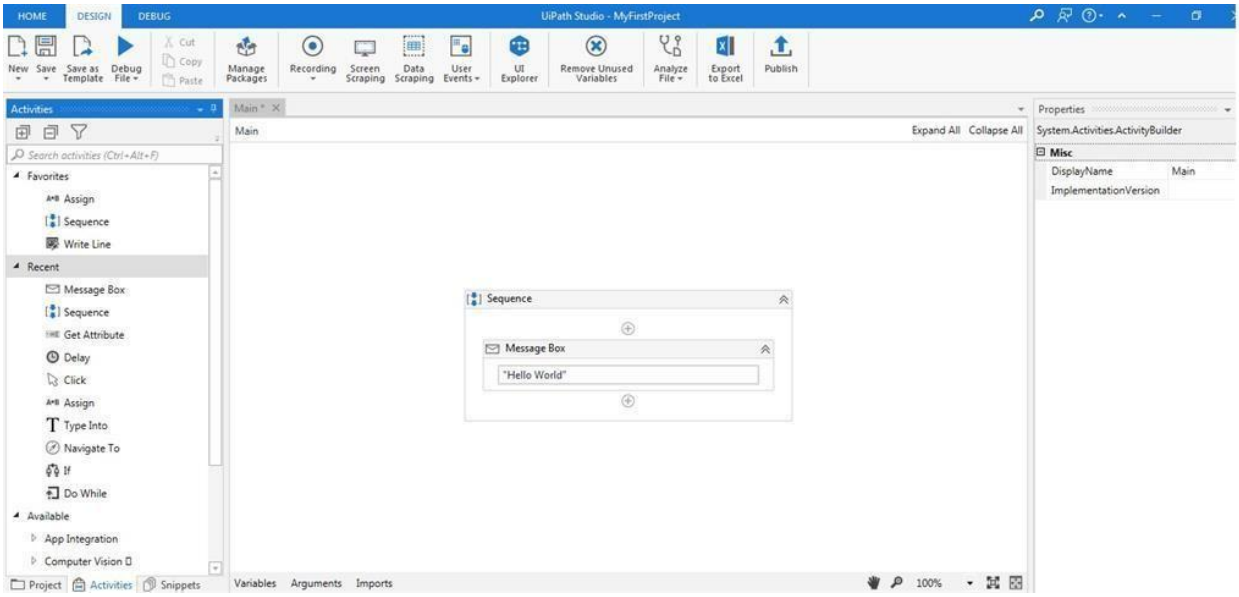
After creating the project:

- Double-click **Main.xaml**
- The Workflow Designer opens.

Step 3: Add a Sequence







Procedure

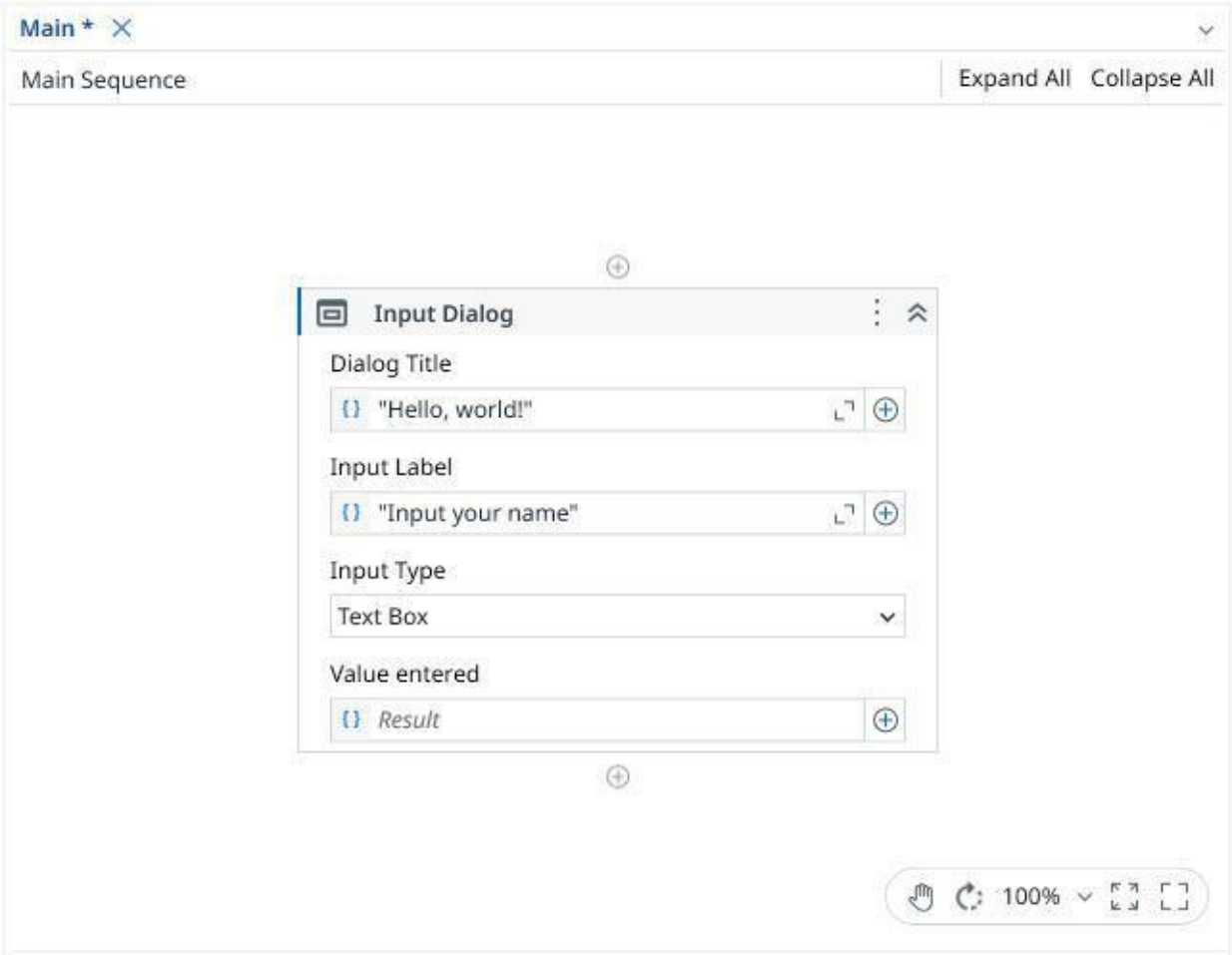
Search

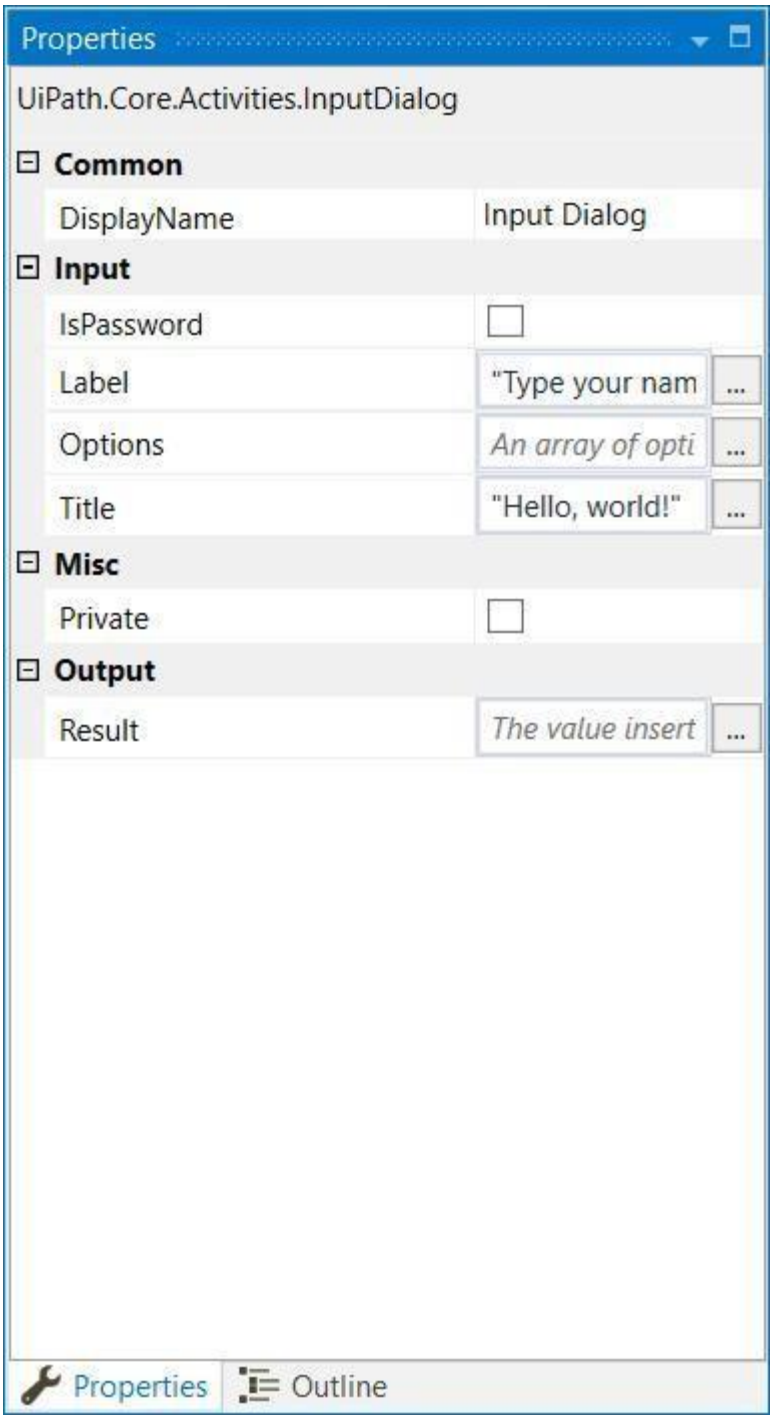
Sequence

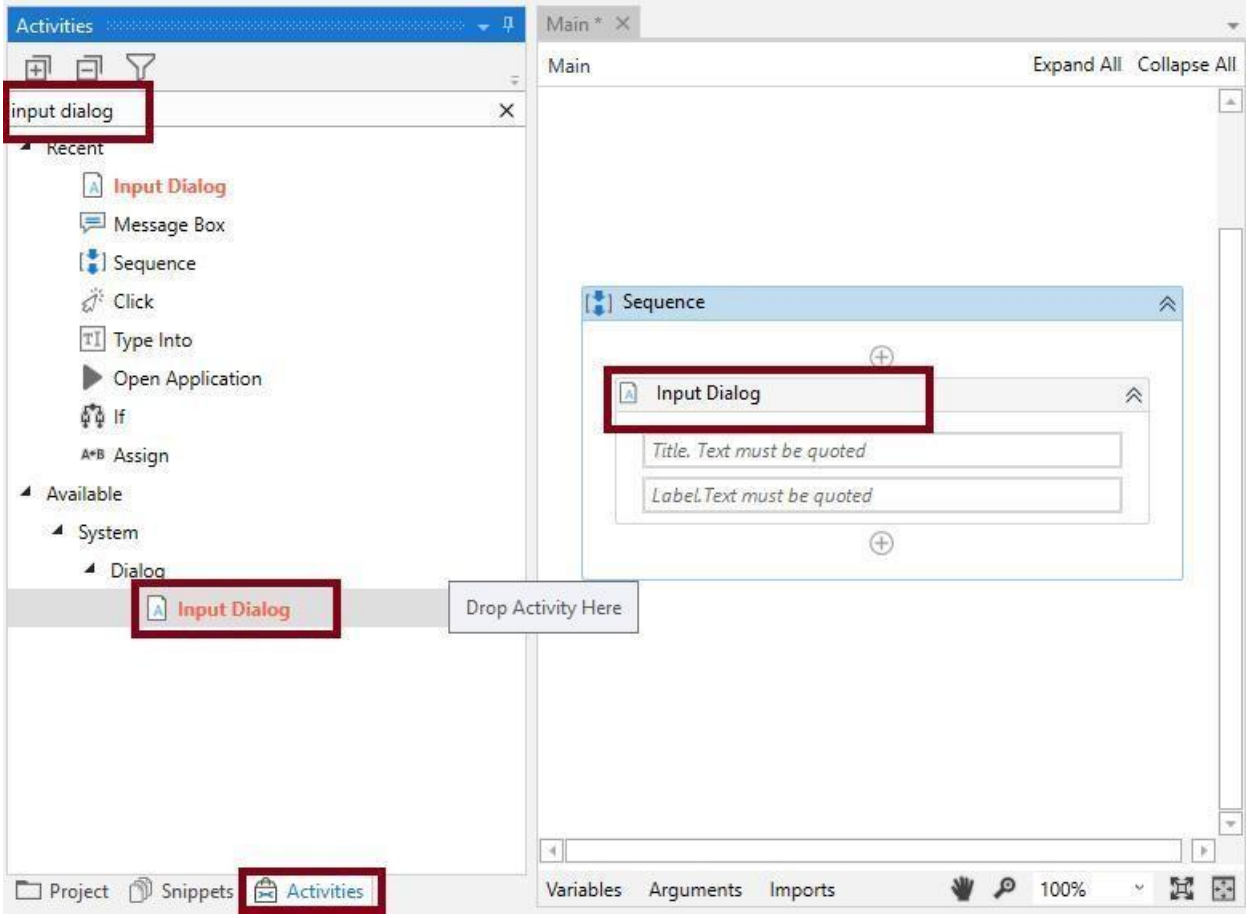
Drag it onto the Designer.

Sequence will contain all the activities.

Step 4: Add Input Dialog


Main *
Main Sequence
Expand All
Collapse All
Input Dialog
Dialog Title
"Hello, world!"
Input Label
"Input your name"
Input Type
Text Box
Value entered
Result
100%


Properties
UiPath.Core.Activities.InputDialog
Common
DisplayName
Input Dialog
Input
IsPassword
Label
"Type your nam
Options
An array of opti
Title
"Hello, world!"
Misc
Private
Output
Result
The value insert
Properties
Outline



4

Search

Input Dialog

Drag it inside the Sequence.

Fill:

Title

Attendance

Label

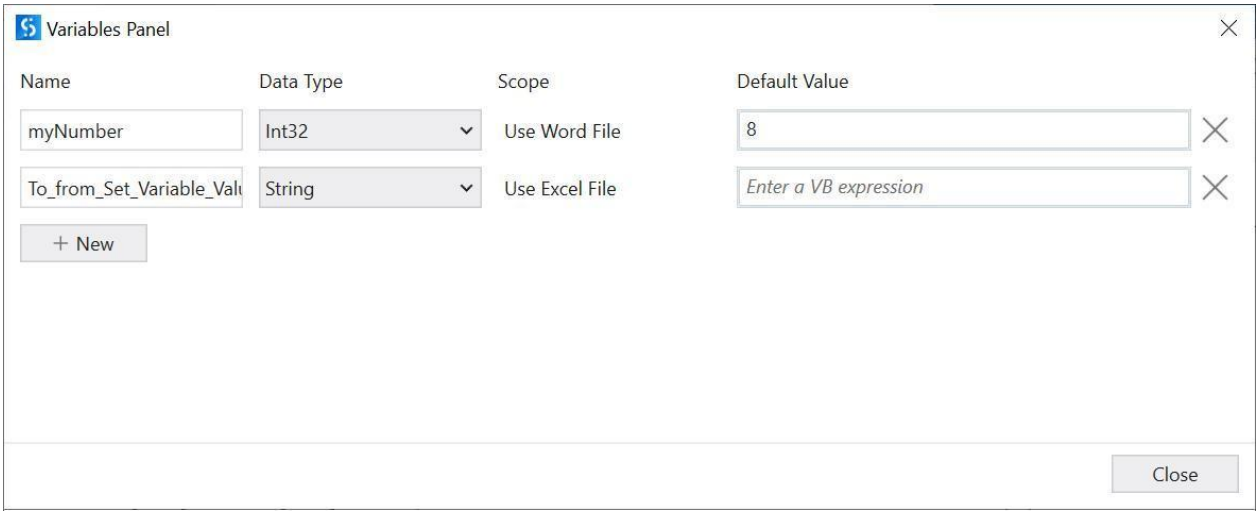
Enter Attendance Percentage

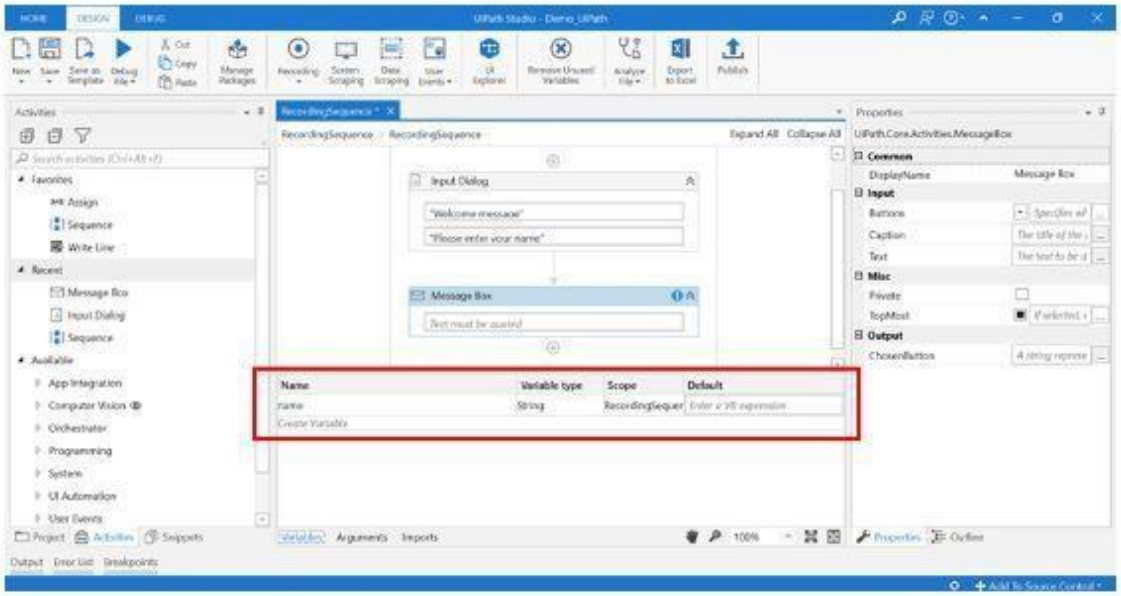
Output Variable

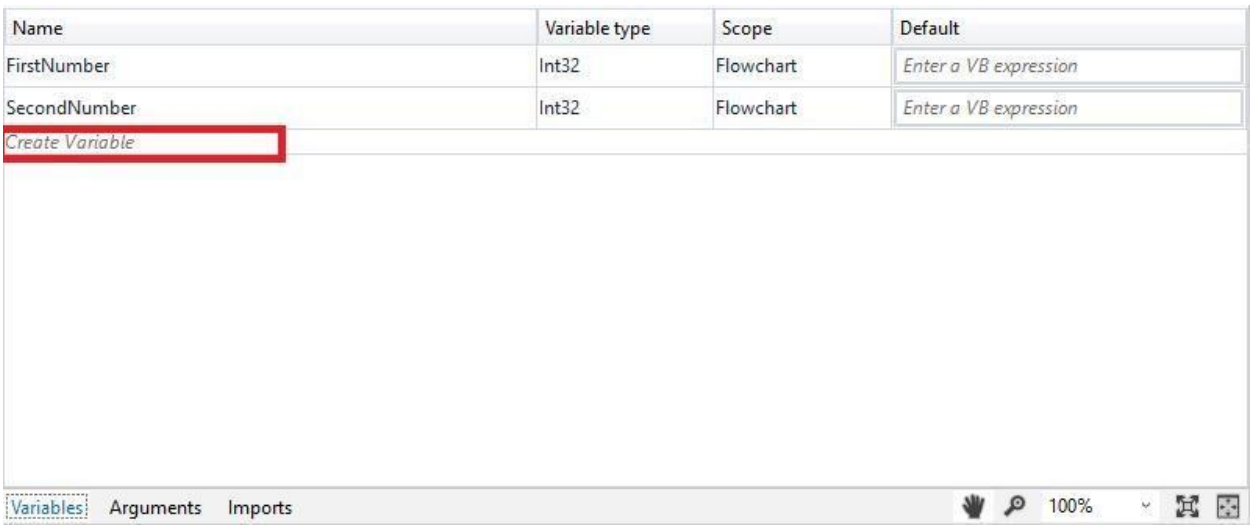
Attendance

(String)

Step 5: Create Variables





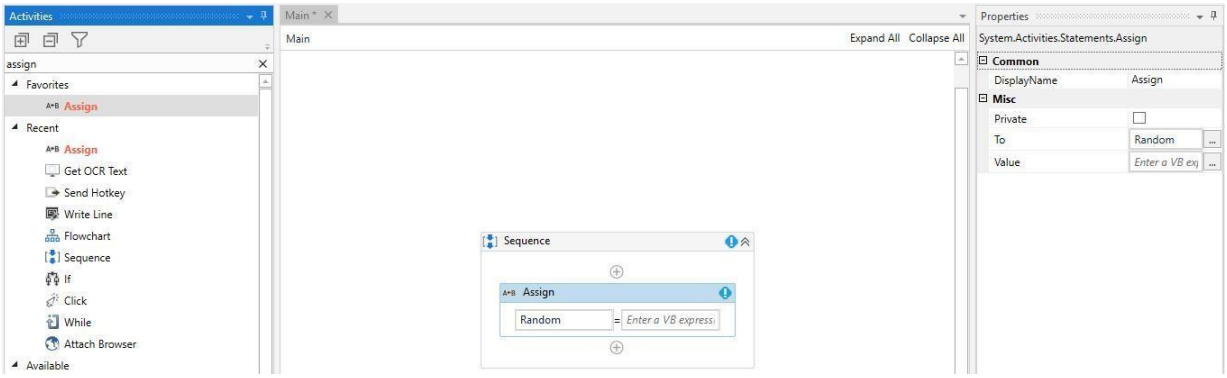


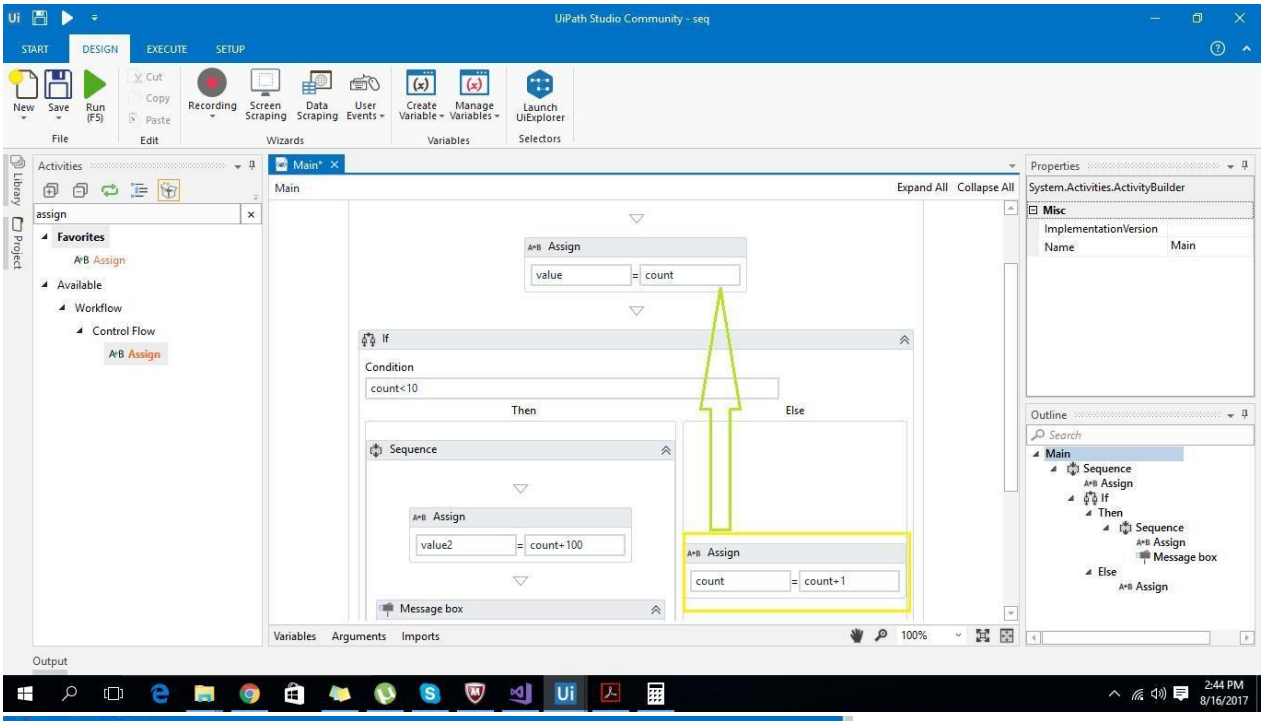
4

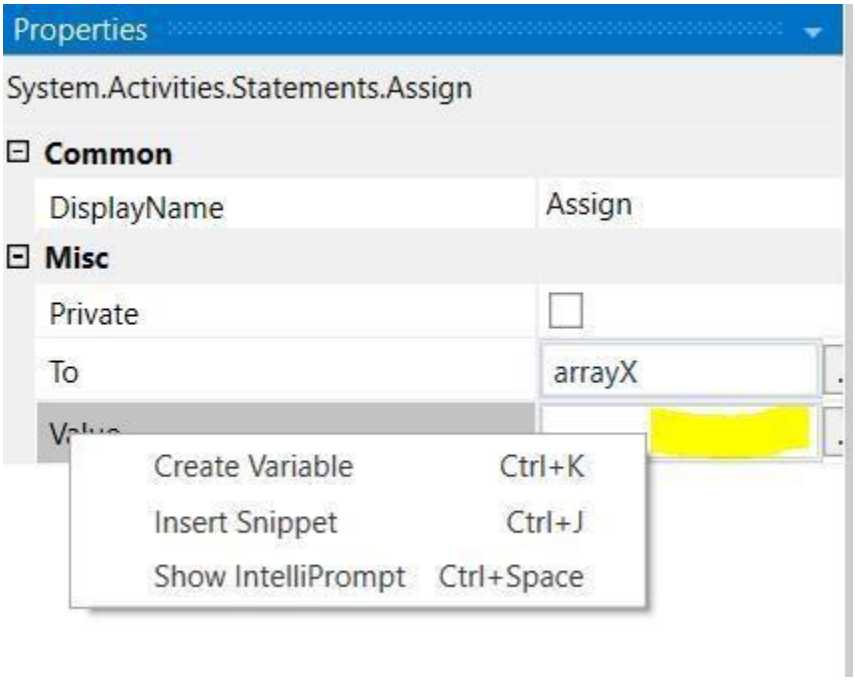
Create two variables.

Variable	Type
Attendance	String
AttendanceInt	Int32

Step 6: Convert String into Integer







Search

Assign

Configure:

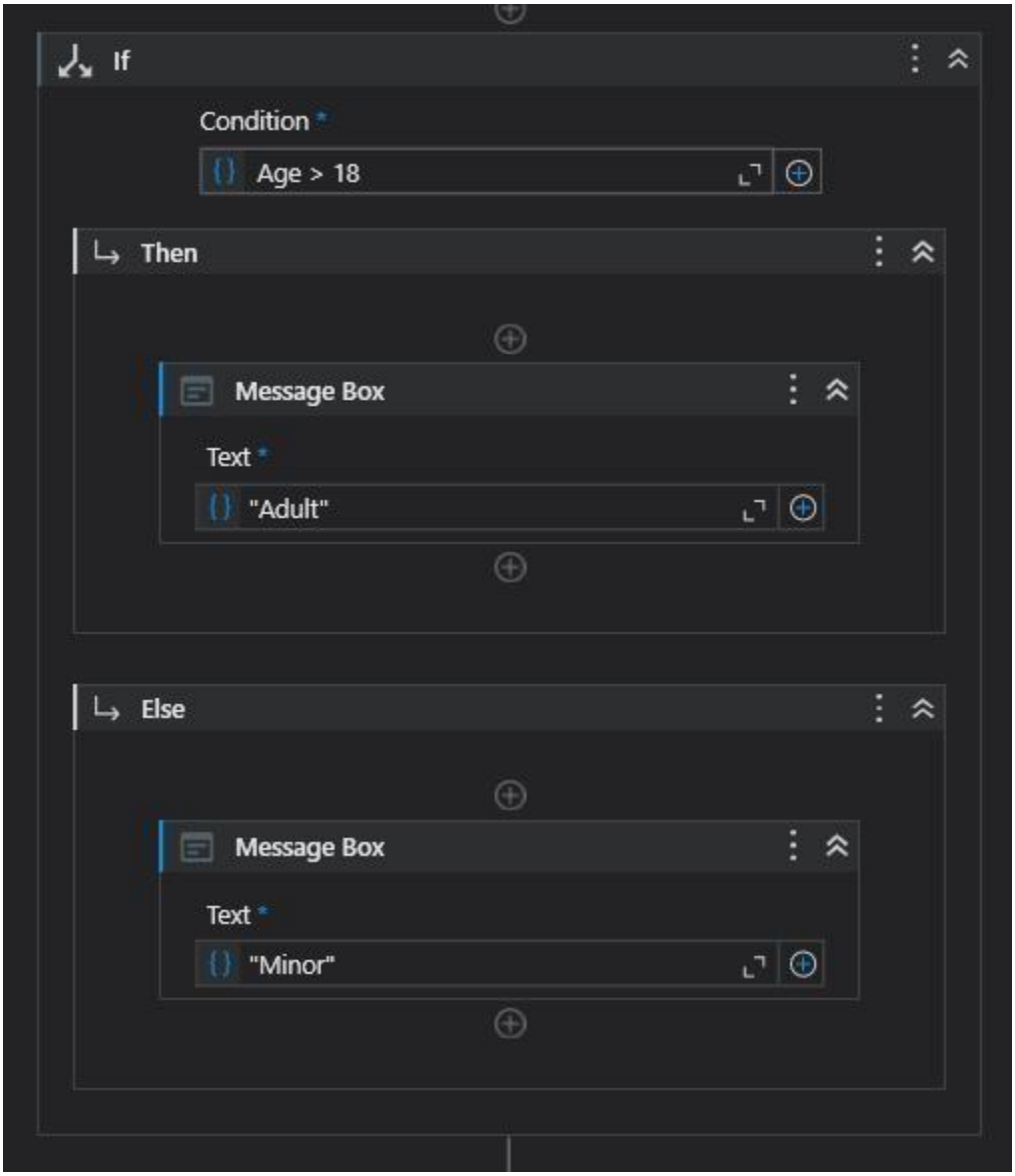
Left

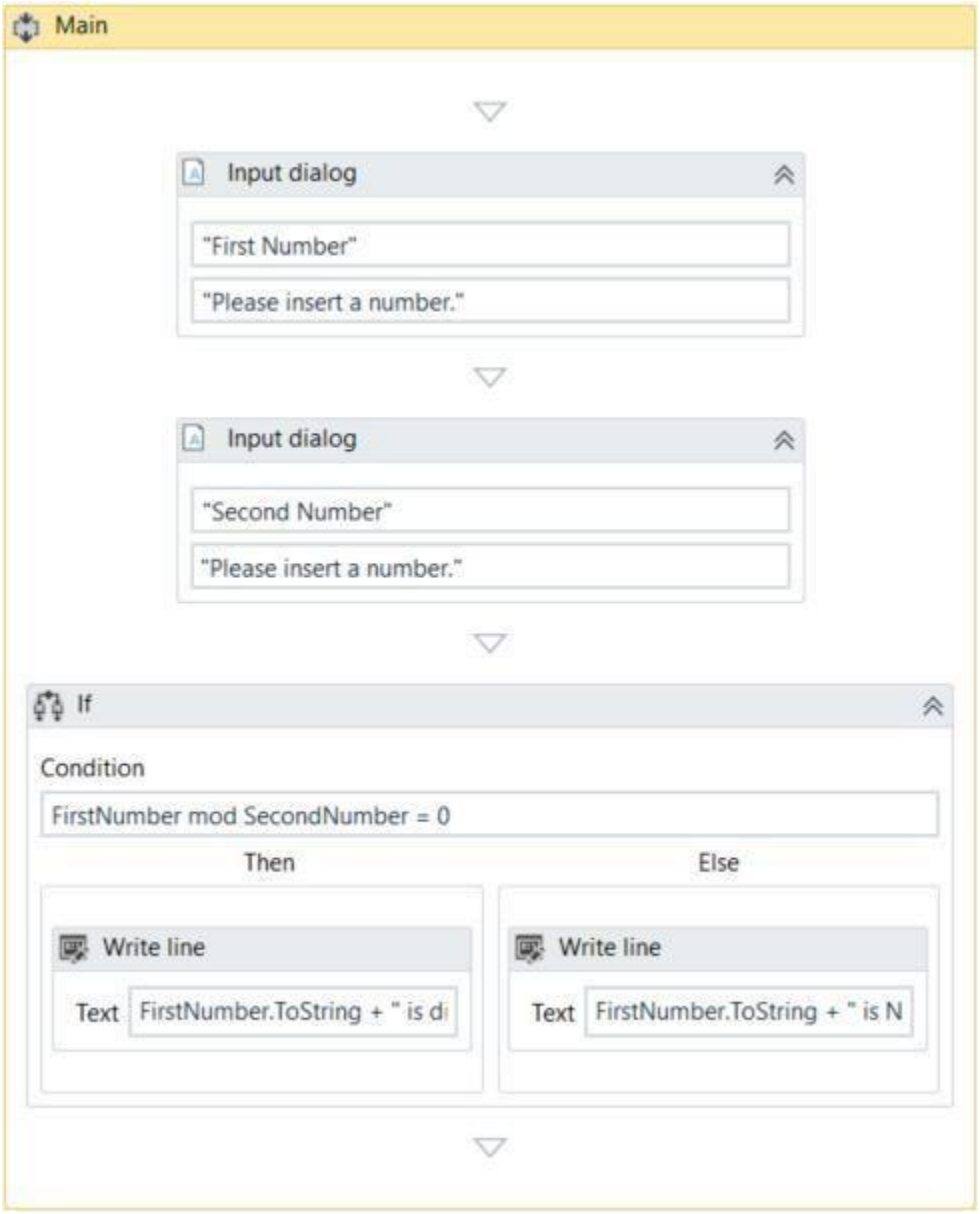
AttendanceInt

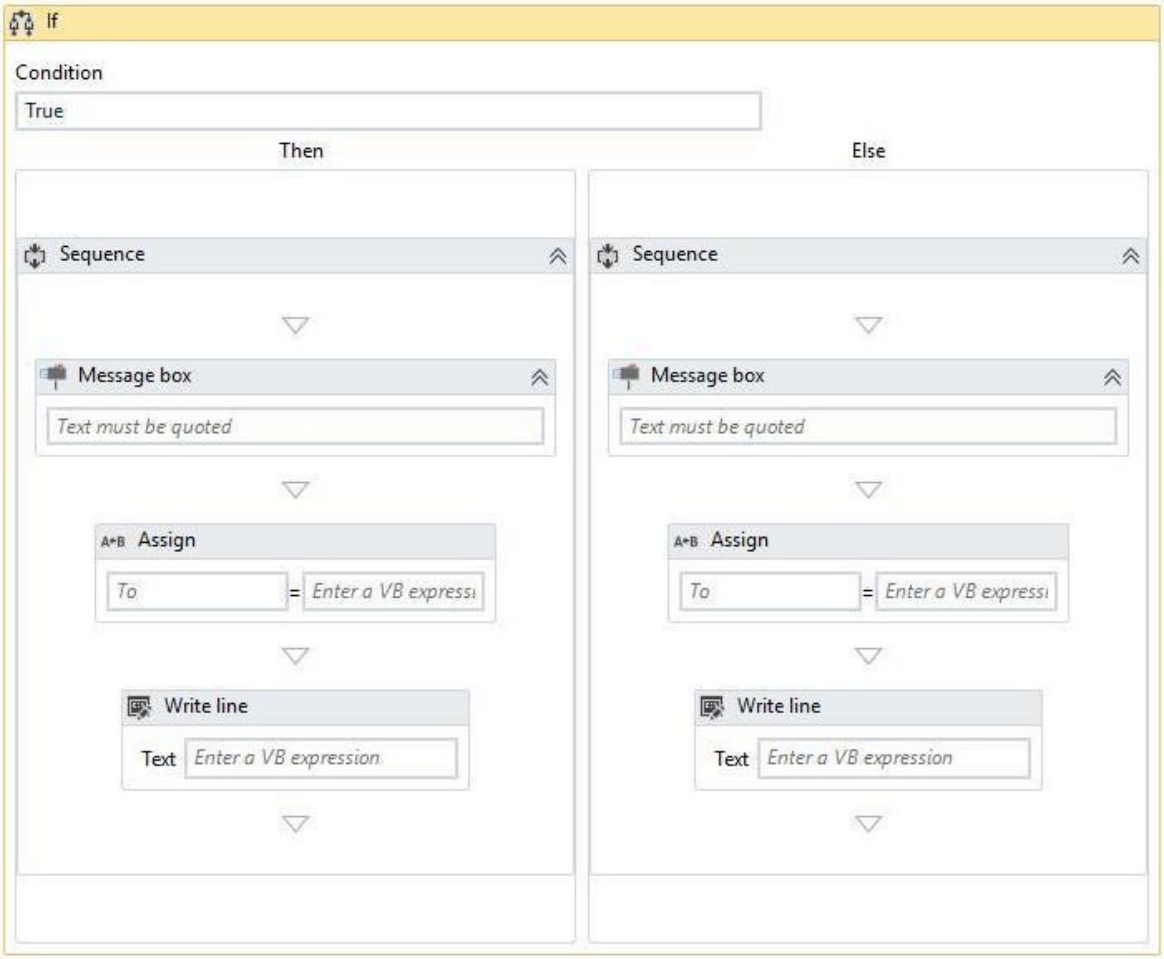
Right

Convert.ToInt32(Attendance)

Step 7: Add If Activity




Main
Input dialog
"First Number"
"Please insert a number."
Input dialog
"Second Number"
"Please insert a number."
If
Condition
FirstNumber mod SecondNumber = 0
Then
Else
Write line
Text
FirstNumber.ToString + " is di
Write line
Text
FirstNumber.ToString + " is N



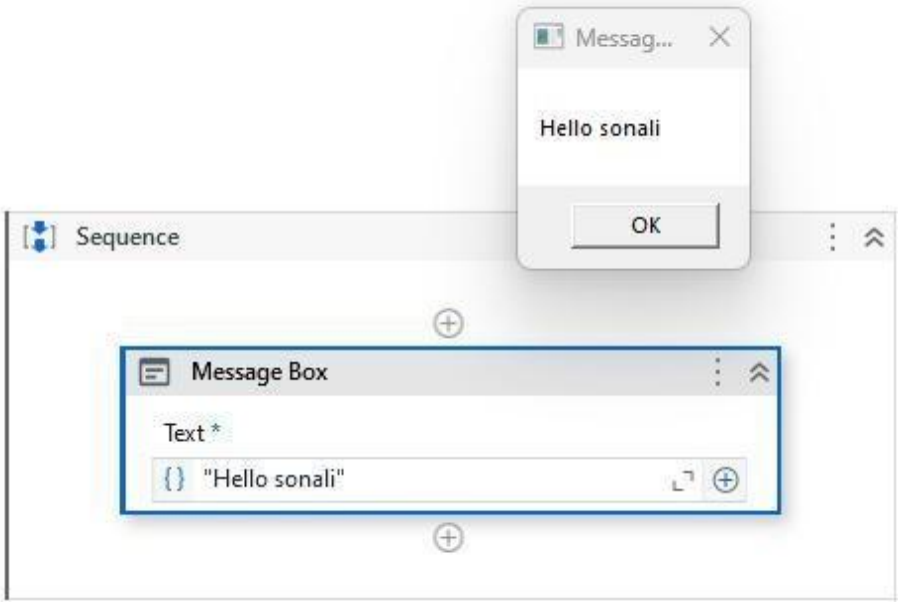
Search

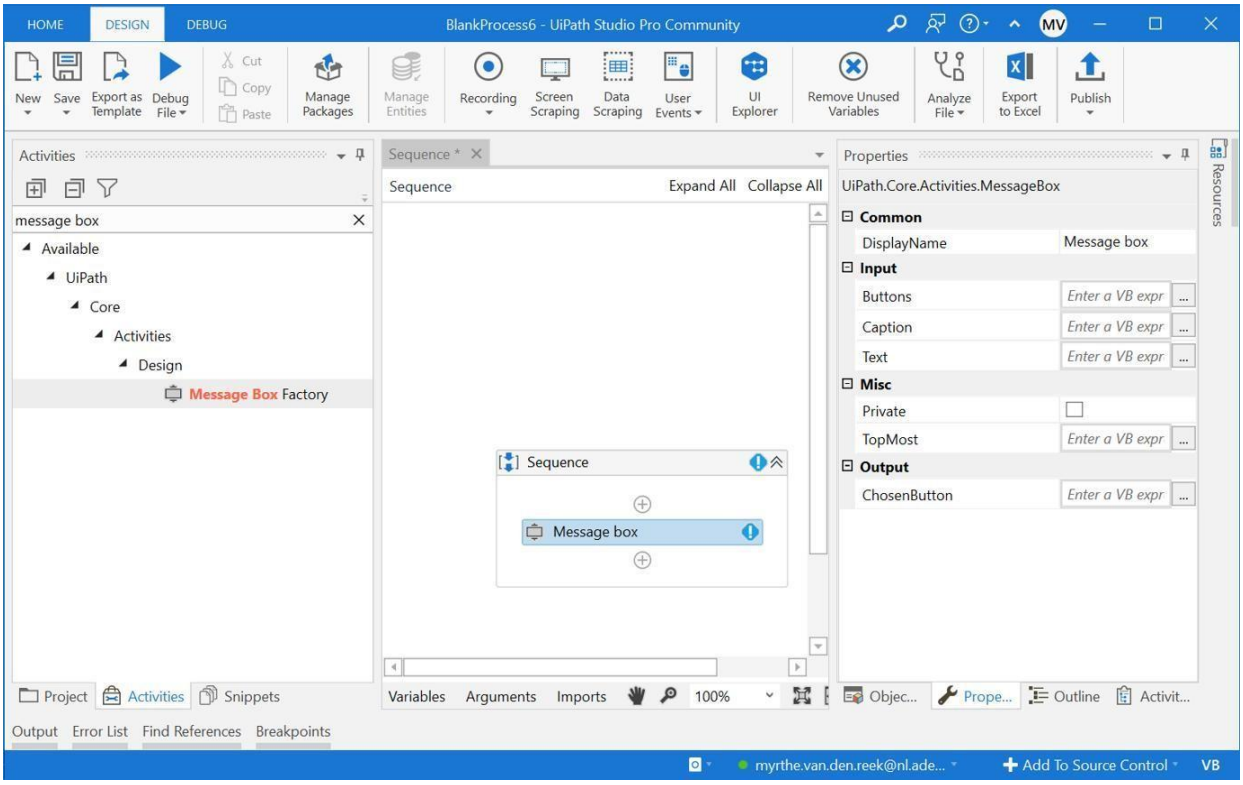
If

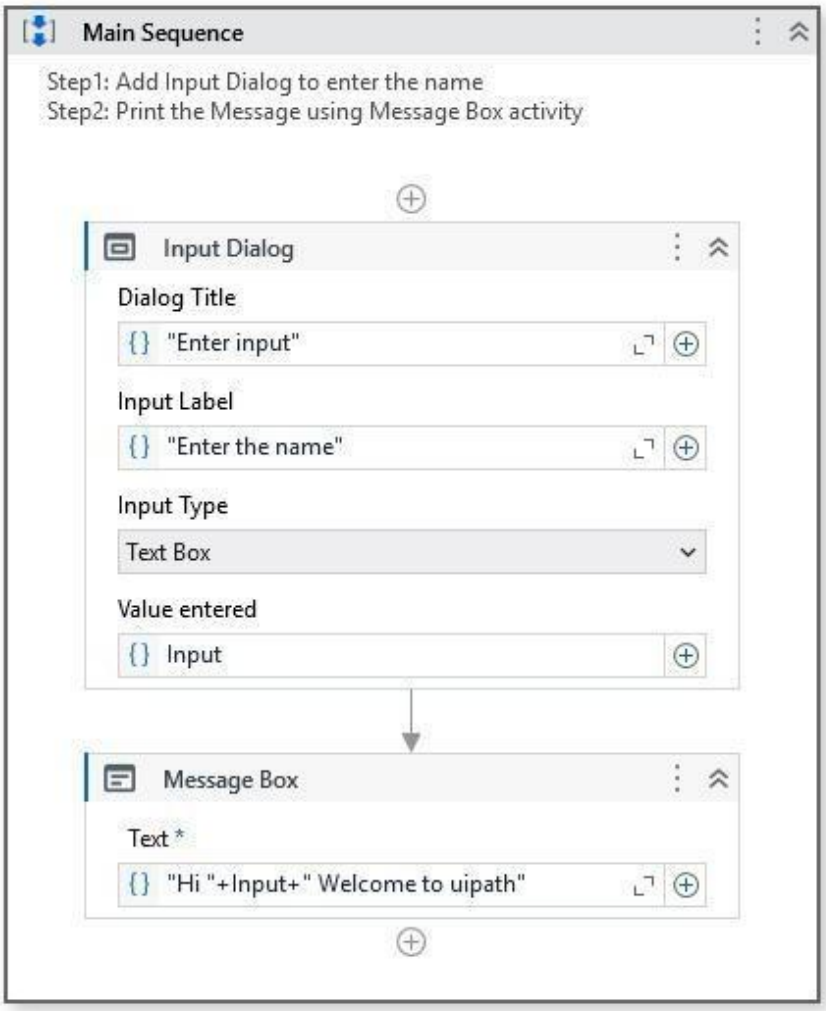
Condition

AttendanceInt >= 75

Step 8: Then Block




HOME
DESIGN
DEBUG
BlankProcess6 - UiPath Studio Pro Community
MV
New
Save
Export as Template
Debug File
Cut
Copy
Paste
Manage Packages
Manage Entities
Recording
Screen Scraping
Data Scraping
User Events
UI Explorer
Remove Unused Variables
Analyze File
Export to Excel
Publish
Activities
message box
Available
UiPath
Core
Activities
Design
Message Box Factory
Sequence *
Sequence
Expand All
Collapse All
Sequence
Message box
Variables
Arguments
Imports
100%
Properties
UiPath.Core.Activities.MessageBox
Common
DisplayName
Message box
Input
Buttons
Caption
Text
Enter a VB expr
Misc
Private
TopMost
Output
ChosenButton
Resources
Project
Activities
Snippets
Objec...
Prope...
Outline
Activit...
Output
Error List
Find References
Breakpoints
myrthe.van.den.reek@nl.ade...
Add To Source Control
VB



5

Inside **Then**

Add

Message Box

Text

Eligible for Examination

Step 9: Else Block

Again add

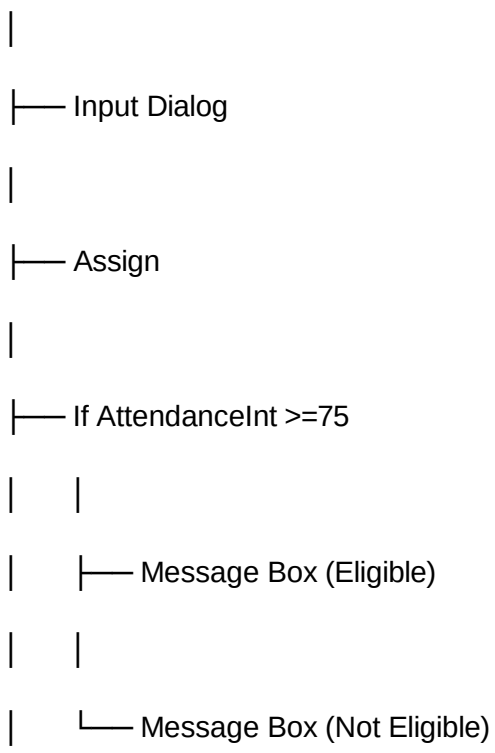
Message Box

Text

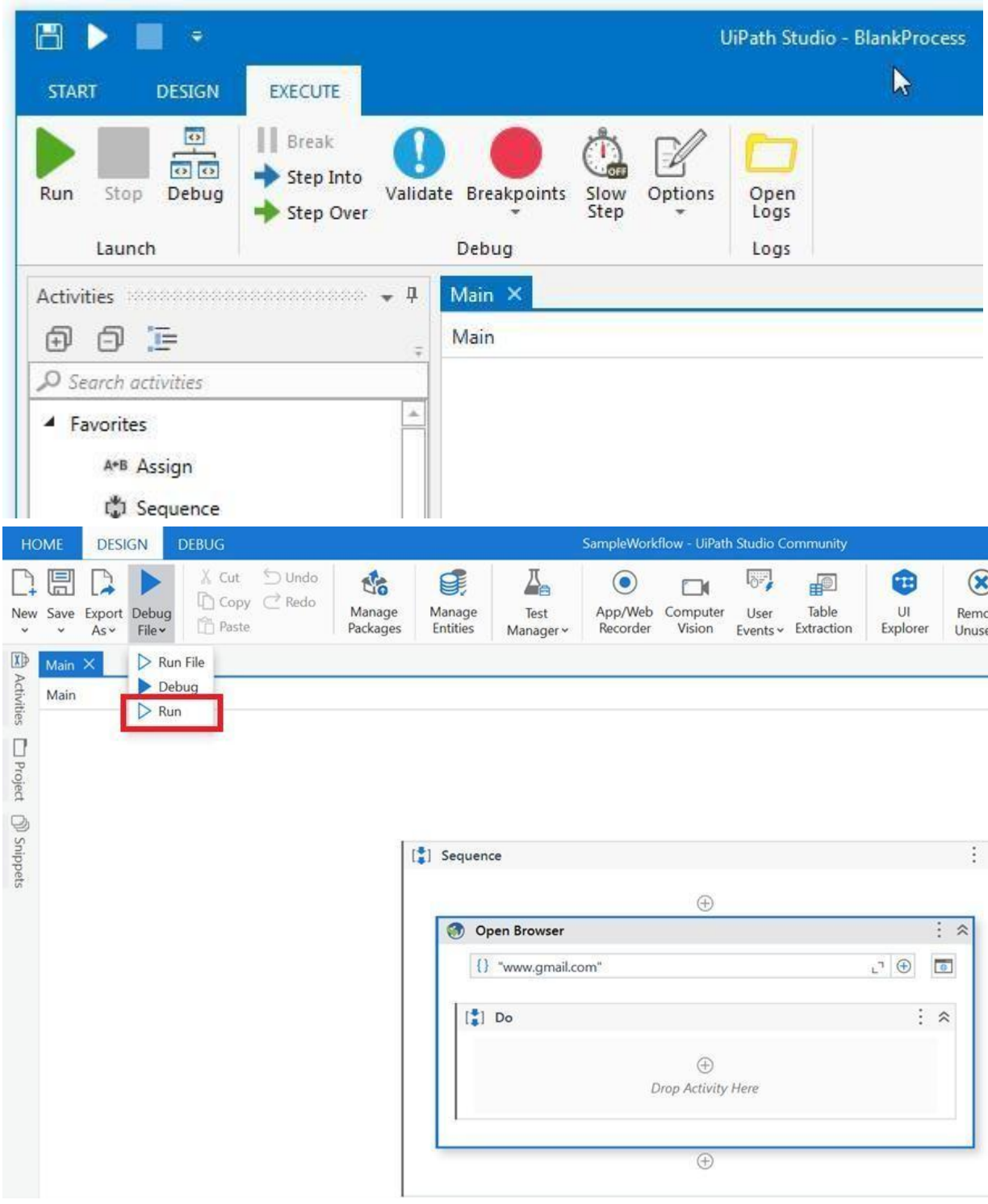
Not Eligible for Examination

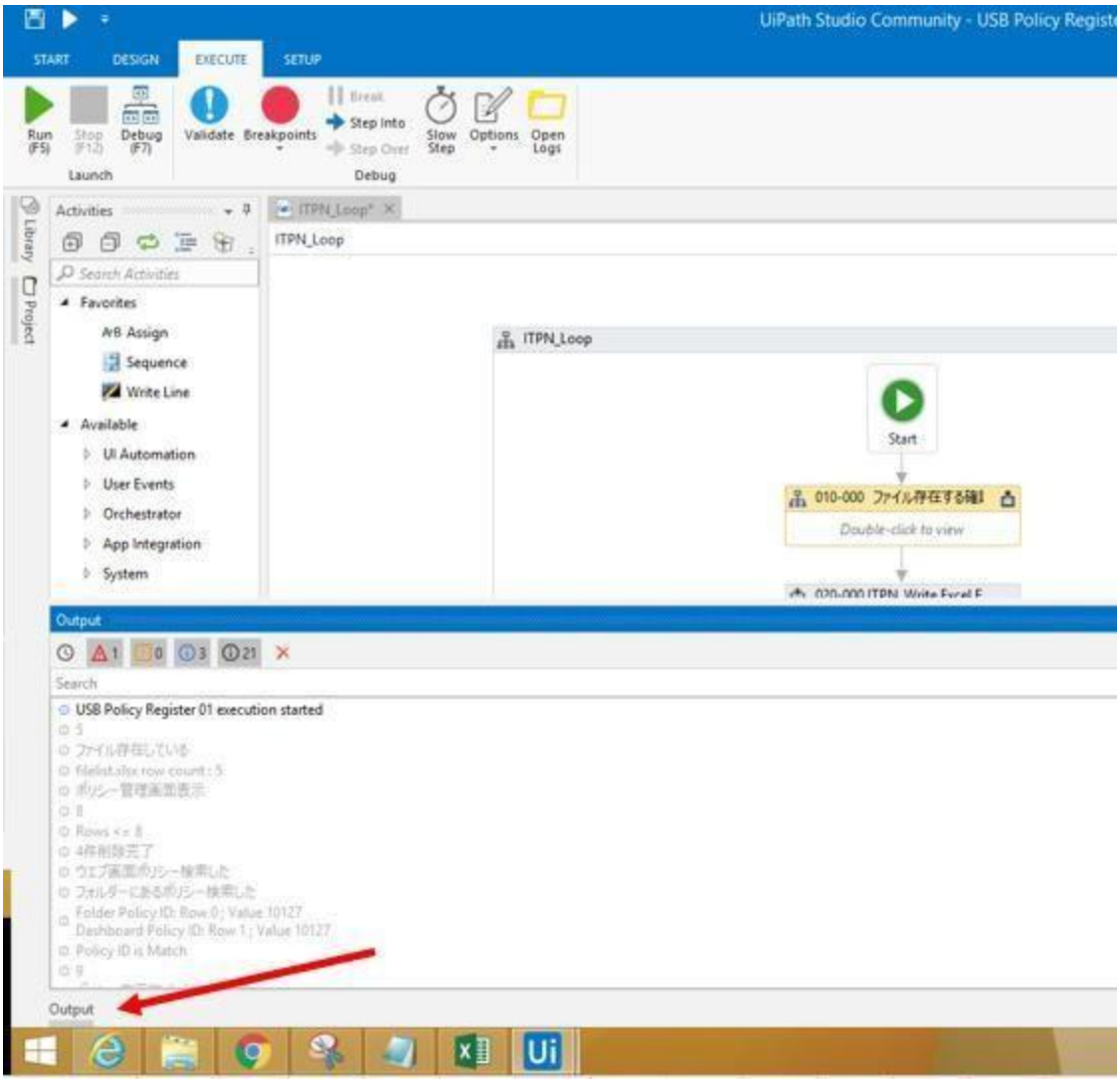
Final Workflow

Sequence



Step 10: Run


UiPath Studio - BlankProcess
START
DESIGN
EXECUTE
Run
Stop
Debug
Launch
Break
Step Into
Step Over
Validate
Breakpoints
Slow Step
Options
Debug
Open Logs
Logs
Activities
Search activities
Favorites
Assign
Sequence
Main
Main
HOME
DESIGN
DEBUG
SampleWorkflow - UiPath Studio Community
New
Save
Export As
Debug File
Cut
Copy
Paste
Undo
Redo
Manage Packages
Manage Entities
Test Manager
App/Web Recorder
Computer Vision
User Events
Table Extraction
UI Explorer
Remove Unused
Run File
Debug
Run
Activities
Project
Snippets
Sequence
Open Browser
"www.gmail.com"
Do
Drop Activity Here



Click

Run

Input

82

Output

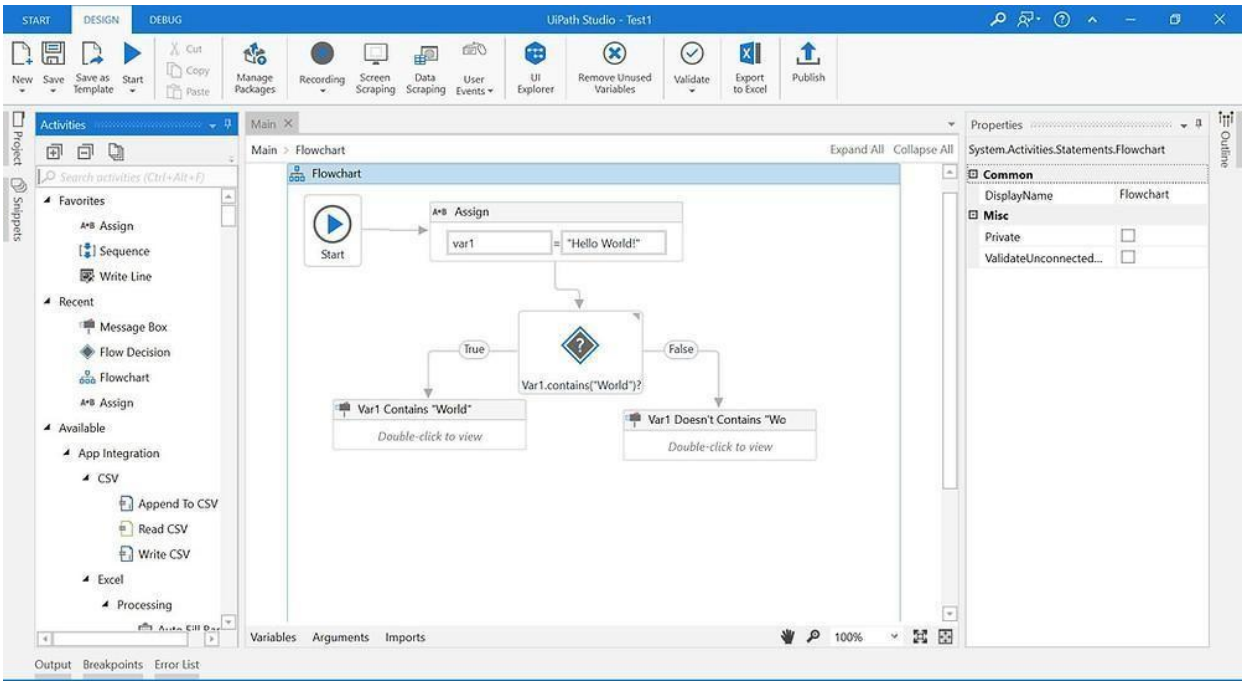
Eligible for Examination

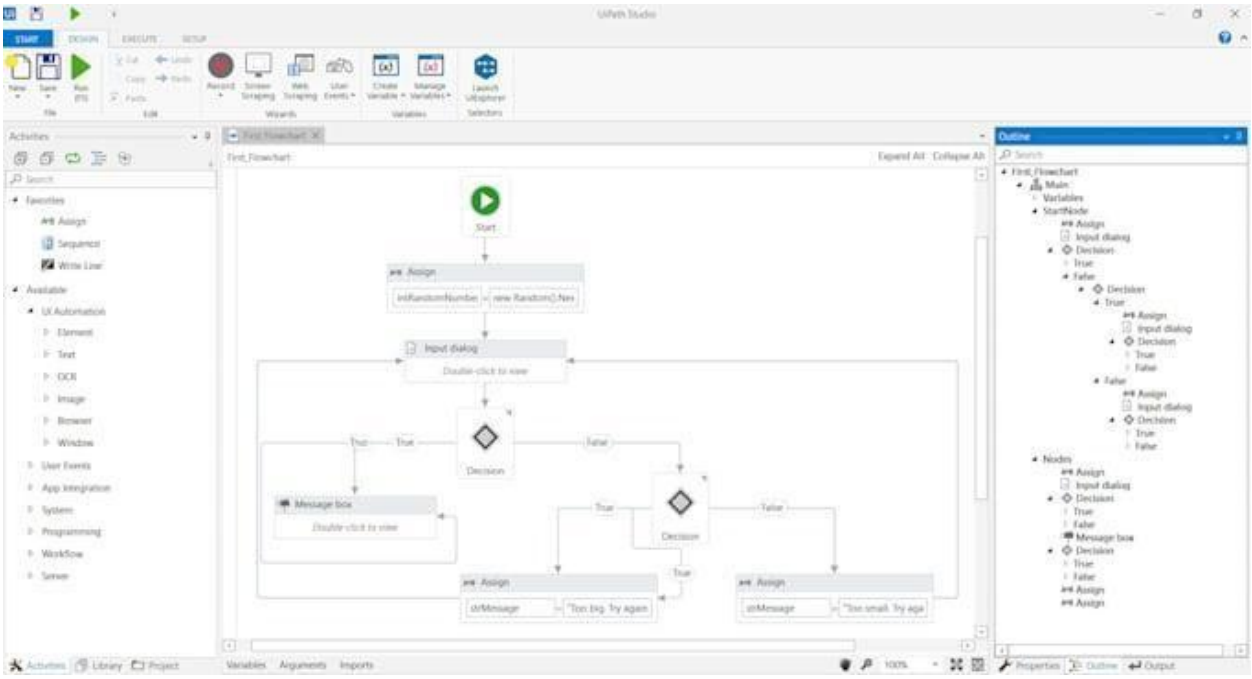
Practical 2: Flowchart Example (Pass or Fail)

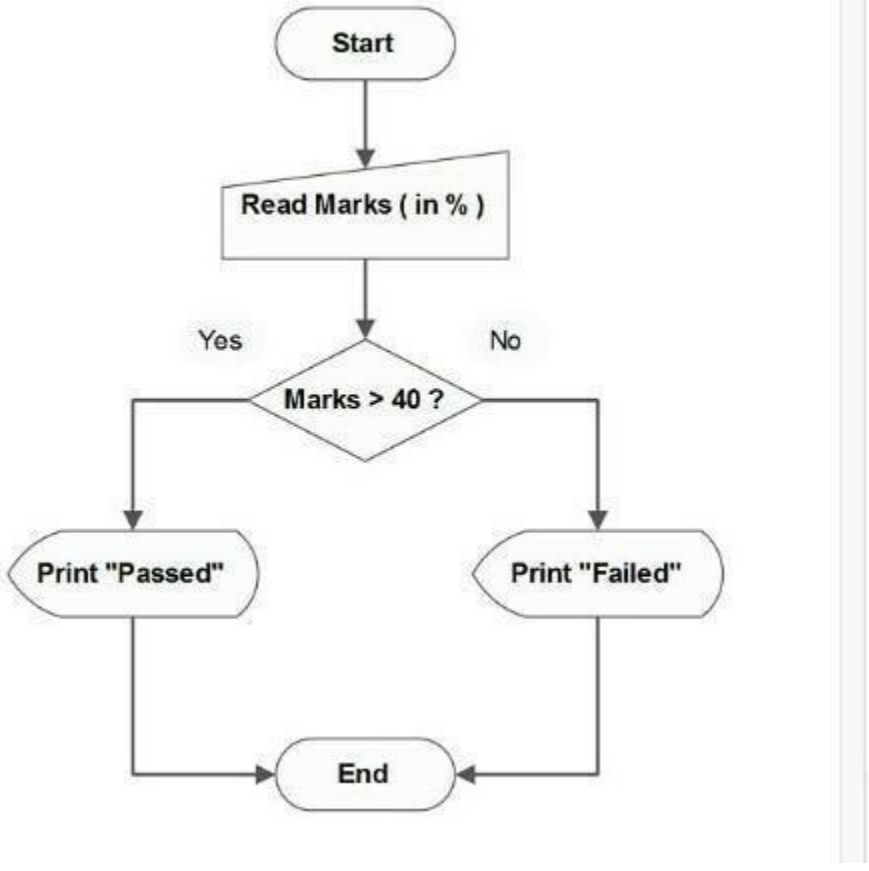
Aim

Develop a Flowchart that determines whether a student has passed.

Flowchart






Start
Read Marks (in %)
Yes
No
Marks > 40 ?
Print "Passed"
Print "Failed"
End

4
Start

↓

Input Marks

↓

Marks >=40 ?

↓

Yes No

↓

Pass Fail

↓

End

Activities Used

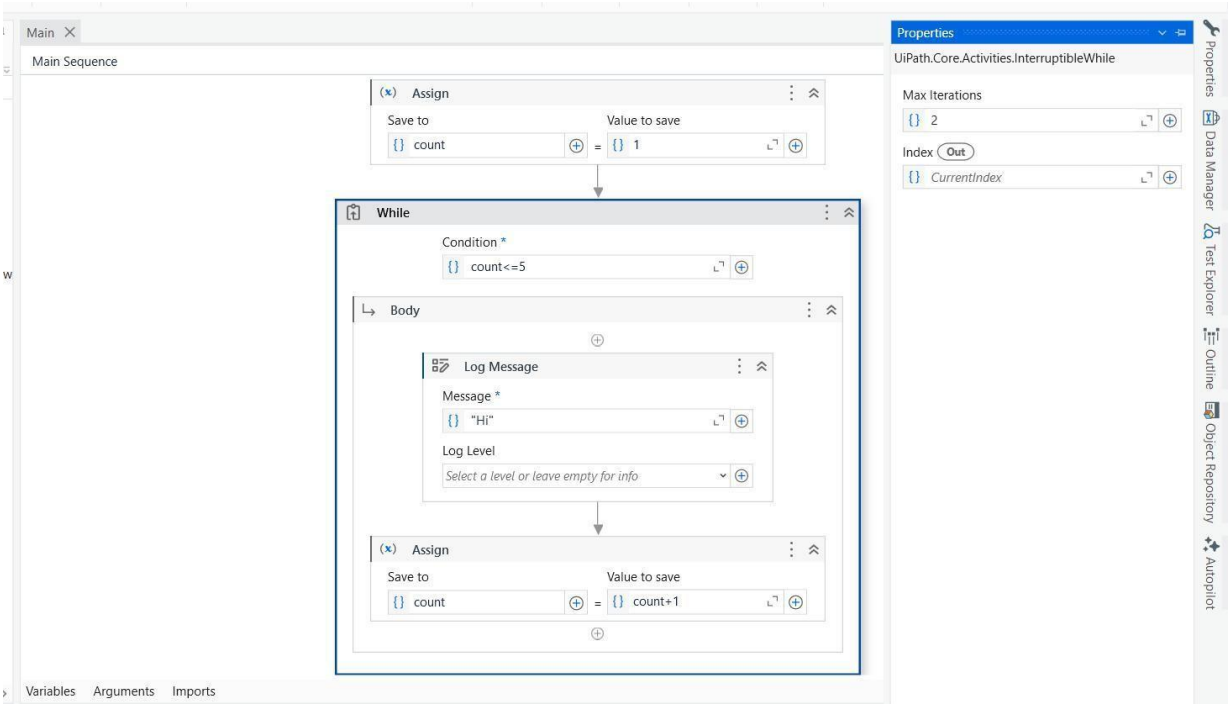
- Flowchart
- Input Dialog
- Assign
- Flow Decision
- Message Box

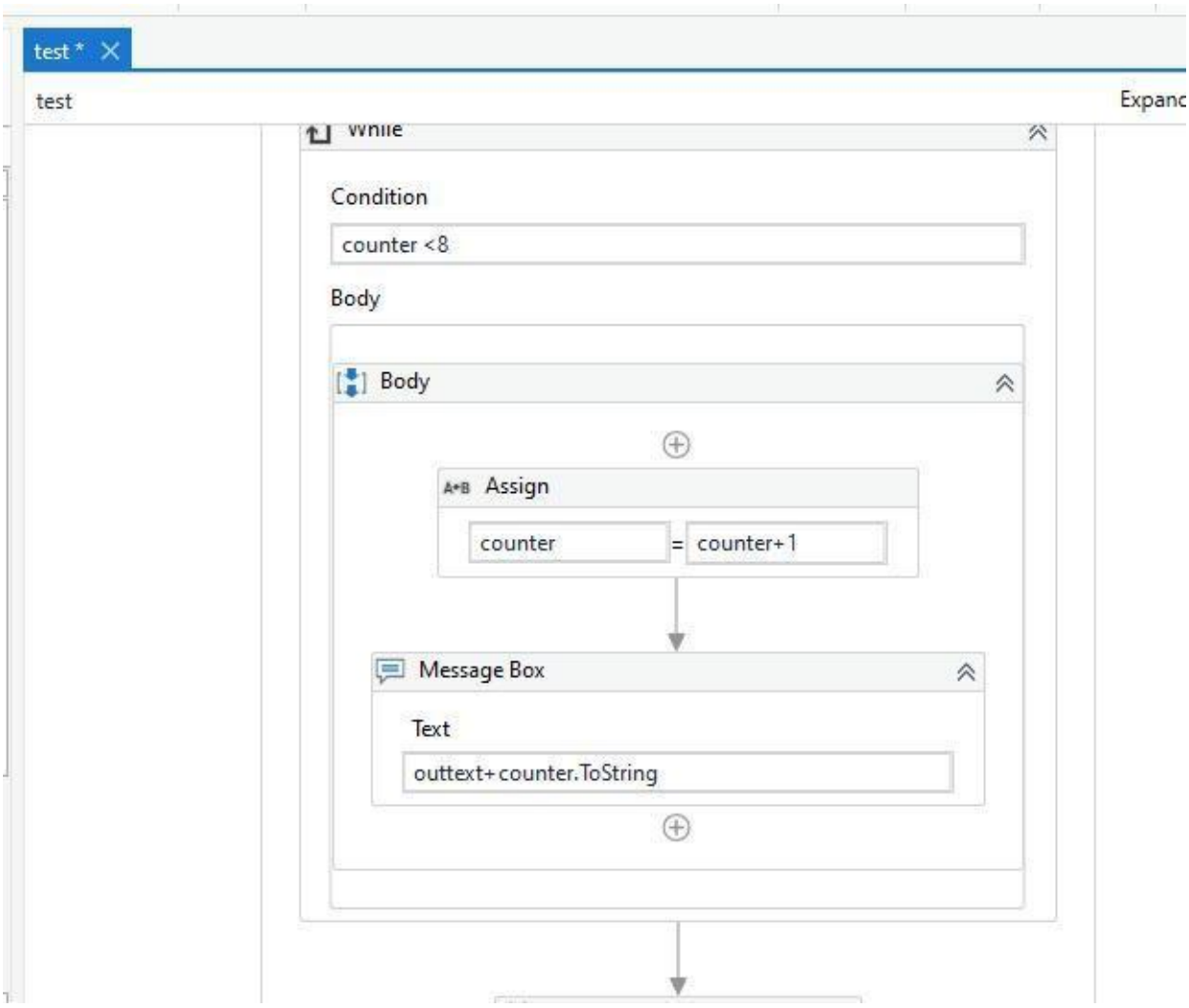
Practical 3: While Loop

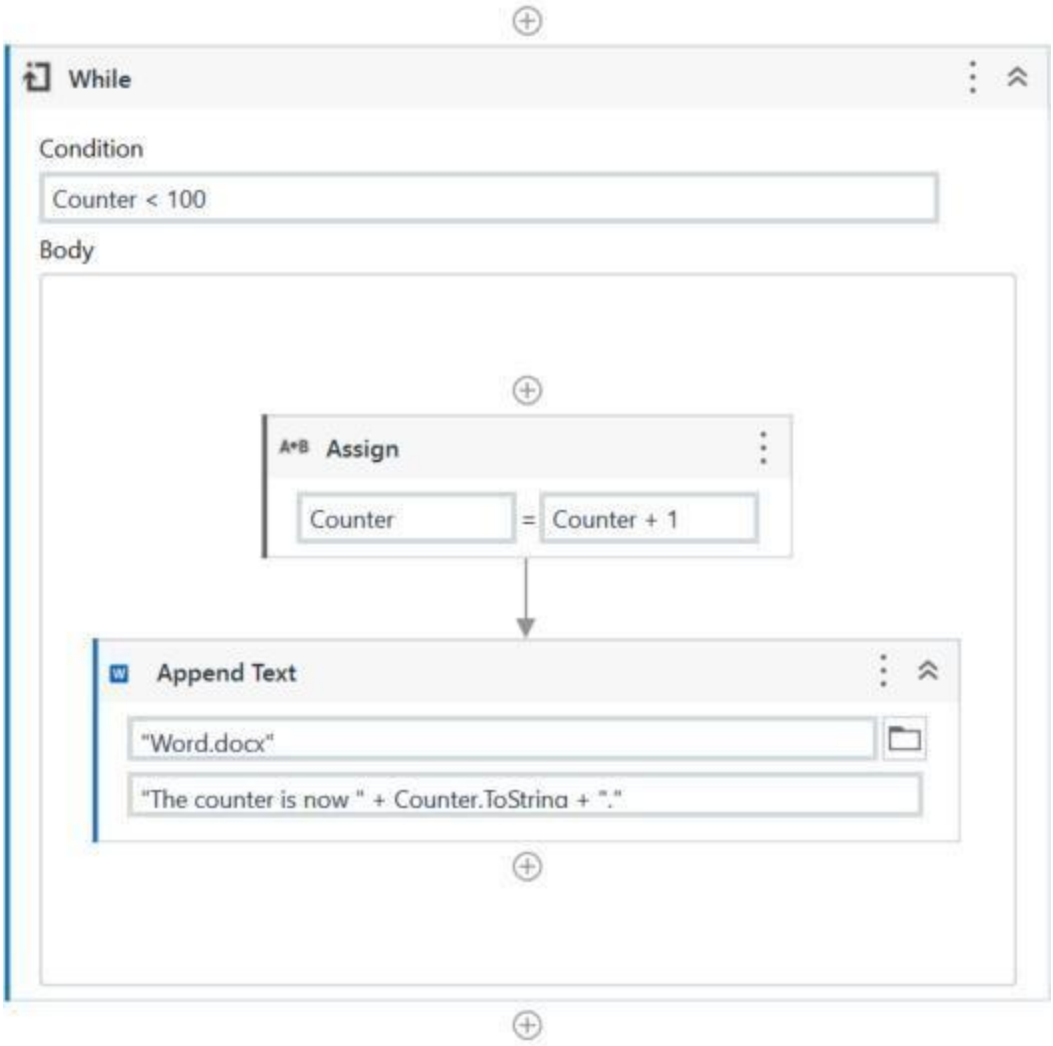
Aim

Print numbers from **1 to 5**

Workflow




test *
test
Expand
While
Condition
counter <8
Body
Body
Assign
counter
=
counter+1
Message Box
Text
outtext+counter.ToString



4
Assign Counter=1

↓

While Counter<=5

↓

Message Box

↓

Counter=Counter+1

↓

End

Output

1

2

3

4

5

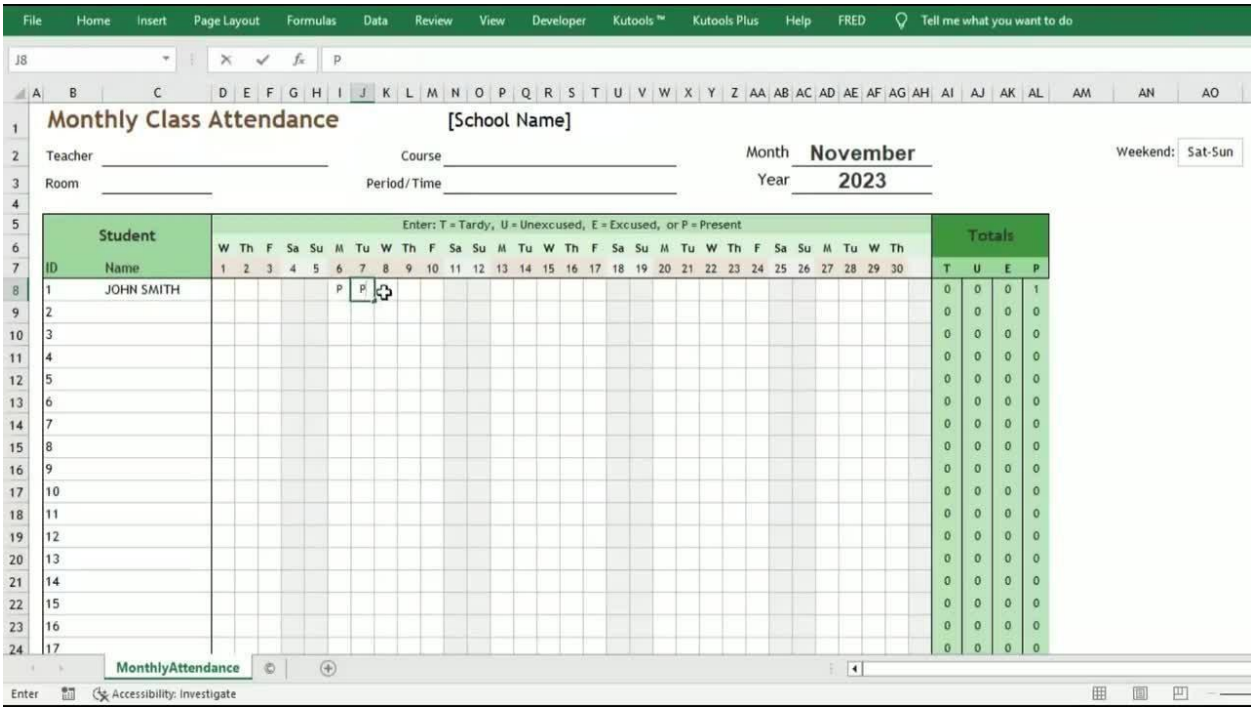
Practical 4: Attendance Automation using Excel

Aim

Read attendance from Excel and determine eligibility.

Excel

	A	B	C	D	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1	Spreadsheet Class.com	Start Date	Today	Holiday	Absent	Present	Excused	(Enter Start Date in E4)				Today's date		8/19		Total Students			12
2					A, 0, Blank	P, 1	E	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat
4	Student Name	Absent	Present	Excused	Holidays	Days Expected	%	8/1	8/2	8/3	8/4	8/5	8/6	8/7	8/8	8/9	8/10	8/11	8/12
5	Name 1	1	12	0	1	13	92.3%	P	P	P	P			P	P	P	P		
6	Name 2	6	7	0	1	13	53.8%	A	P	P	A			P	P	A	A		
7	Name 3	1	11	1	1	12	91.7%	P	E	P	P			P	P	P	P		
8	Name 4	6	7	0	1	13	53.8%	P	A	P	A			A	A	A	P		
9	Name 5	1	12	0	1	13	92.3%	P	P	P	P			P	P	P	P		
10	Name 6	3	8	2	1	11	72.7%	P	P	P	A			P	P	P	A		
11	Name 7	5	8	0	1	13	61.5%	A	P	P	A			P	P	A	A		
12	Name 8	1	11	1	1	12	91.7%	P	P	P	E			P	P	P	P		
13	Name 9	2	11	0	1	13	84.6%	P	P	P	P			P	P	P	P		
14	Name 10	5	8	0	1	13	61.5%	P	A	A	A			P	A	P	P		
15	Name 11	2	11	0	1	13	84.6%	P	P	P	P			P	P	P	P		
16	Name 12	1	12	0	1	13	92.3%	P	P	P	P			P	P	P	P		



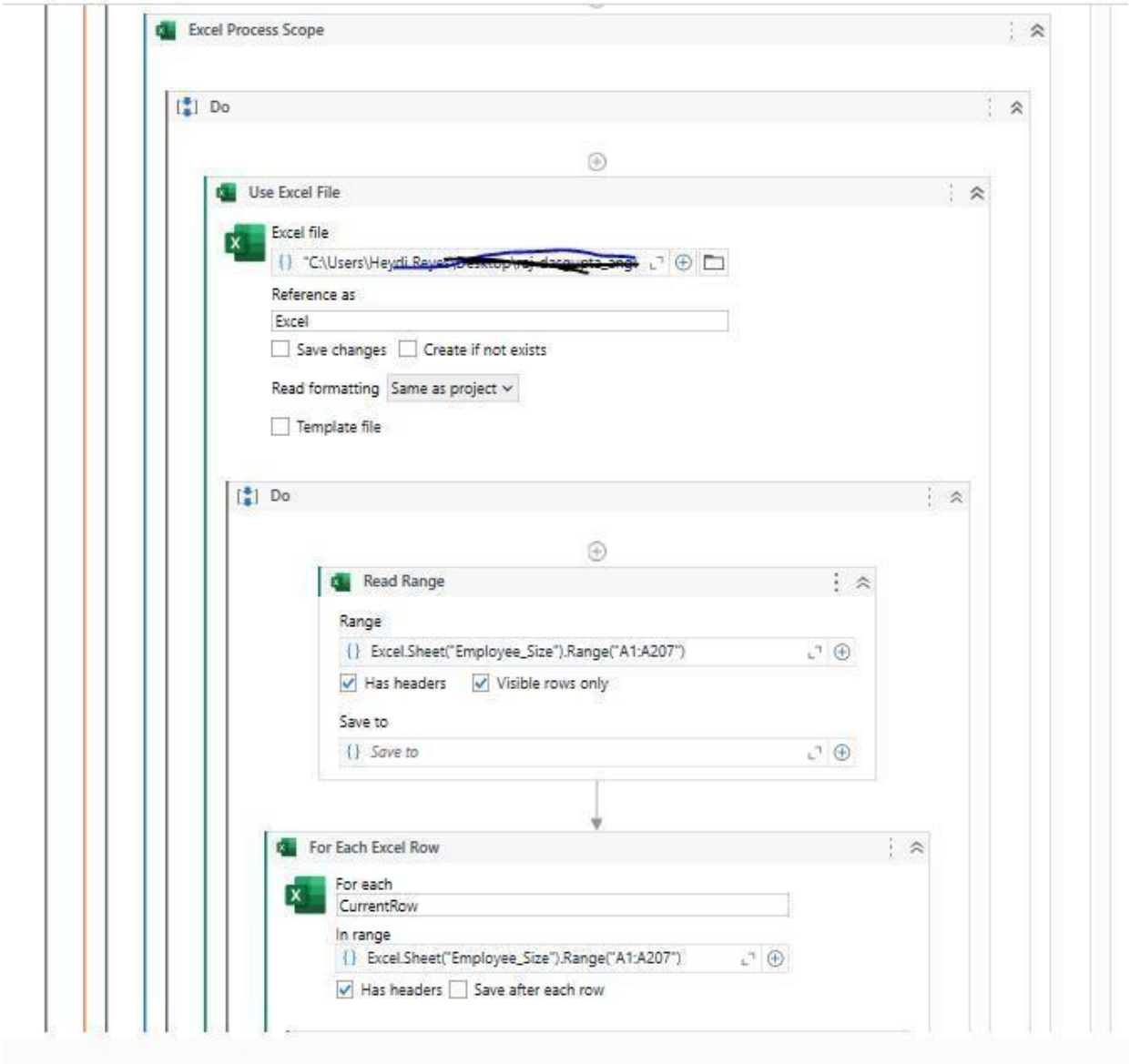
	A	B	C
1	**ID**	**Name**	**Attendance**
2	1	Billy	
3	2	Billy	
4	3	Mcfaden	
5	4	Steven Shwimmer	
6	5	Ruby jason	
7	6	Mark Dyne	
8	7	Philip namdaf	
9	8	Erik Bawn	
10	9	Ricky ben	
11	10	Miecky	
12			
13			
14			
15			
16	Total		
17	Present		
18	Absent		

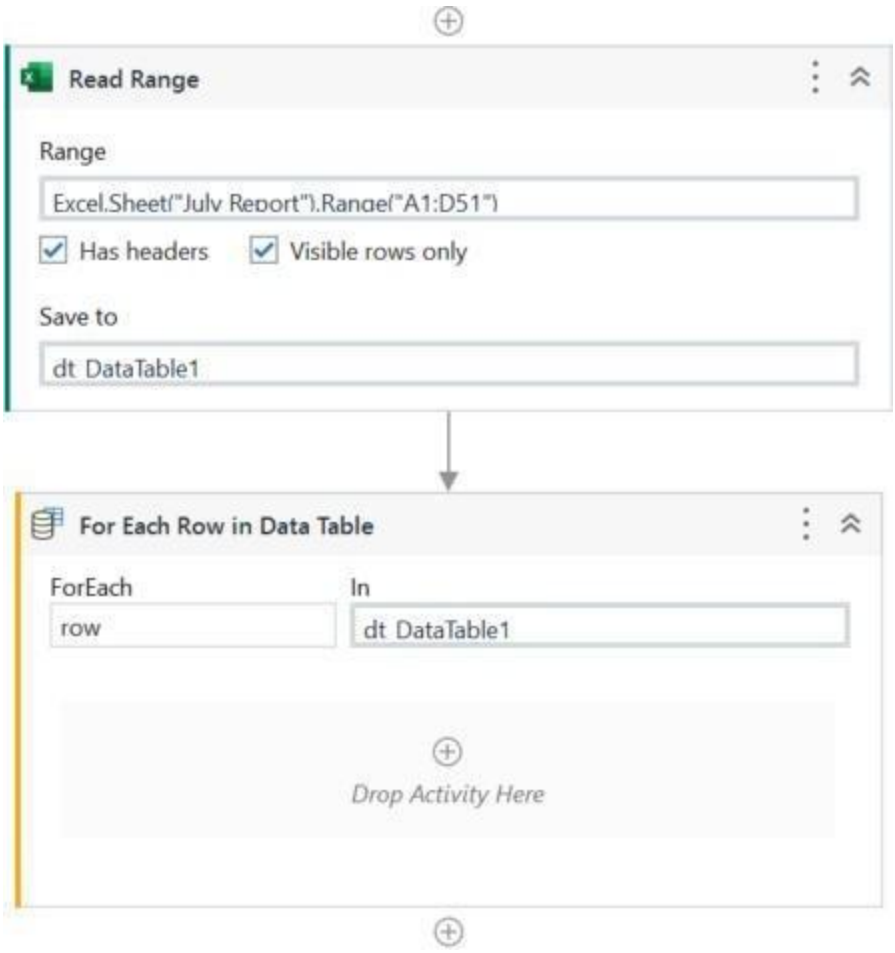
4

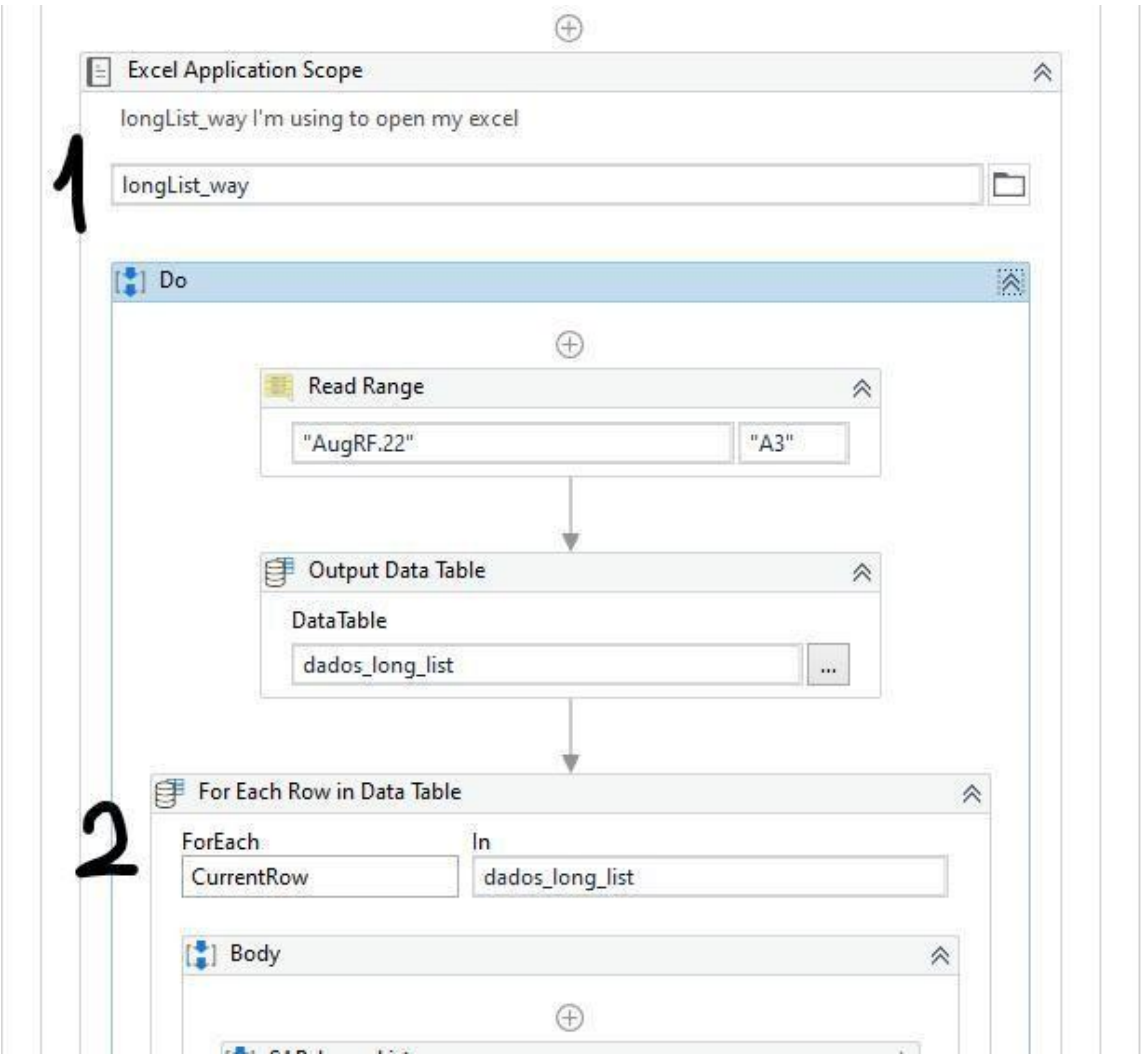
Student **Total Classes** **Attended**

Rahul	100	90
Priya	100	70
Amit	100	82

Workflow




Read Range
Range
Excel.Sheet("July Report").Range("A1:D51")
Has headers
Visible rows only
Save to
dt_DataTable1
For Each Row in Data Table
ForEach
row
In
dt_DataTable1
Drop Activity Here



4

Use Excel File

↓

Read Range

↓

For Each Row

↓

Assign Percentage

↓

If Percentage>=75

↓

Write Cell

↓

End

Activities

- Use Excel File
- Read Range
- For Each Row
- Assign
- If
- Write Cell

Practical 5: Array Processing in UiPath

Aim

To create a UiPath automation workflow that processes an array of strings (names), filters out the names starting with the letter "A" or "a", counts them, and displays the final count to the user.

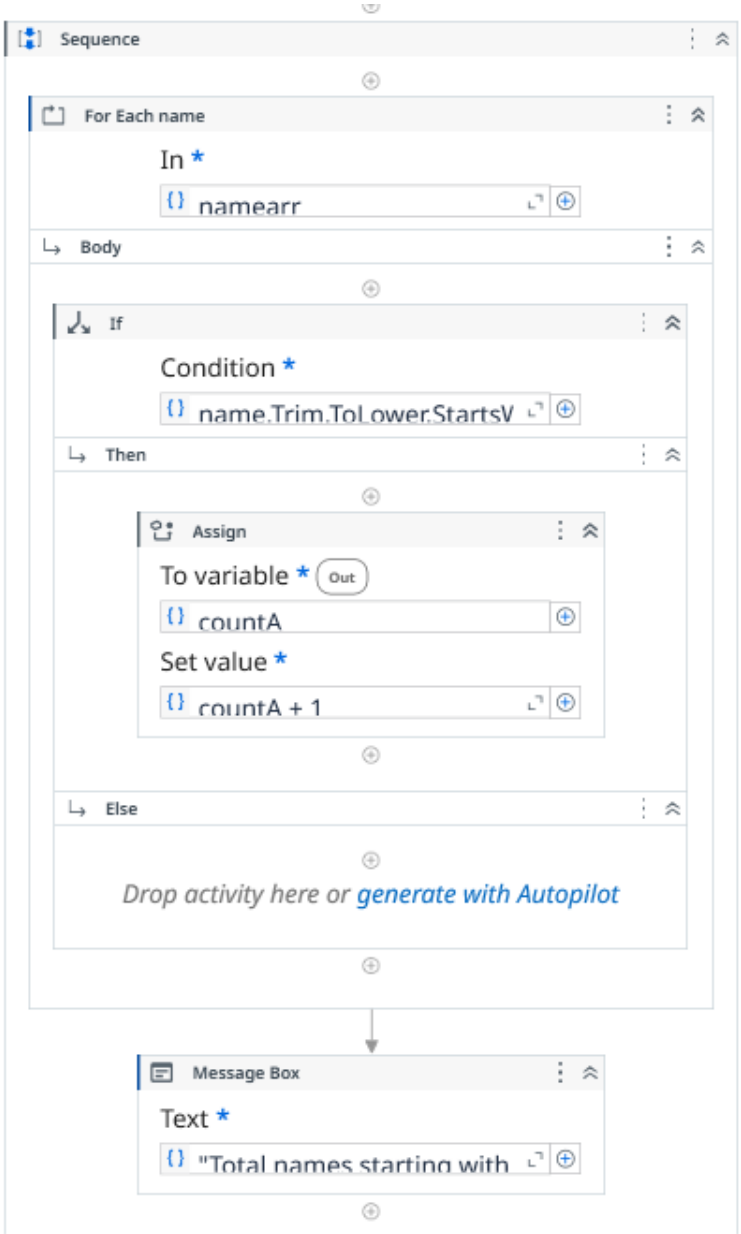
Theory

- **Array:** A data structure that stores a fixed-size sequential collection of elements of the same type. In UiPath, it is represented as `System.String[]` for names.
- **For Each Activity:** This activity allows you to step through the elements of a collection (like an array or list) one by one to perform actions on them.
- **If Activity:** A conditional statement used to evaluate a specific expression. If the expression is true, the activities inside the `Then` block execute; otherwise, the `Else` block executes.
- **Methods Used:**
    - `.StartsWith("a")`: A string method that checks if a piece of text begins with a specific character.
    - `.ToLower` or `.ToUpper`: Methods used to convert strings to a uniform case to prevent case-sensitivity errors (e.g., catching both "Alice" and "alex").

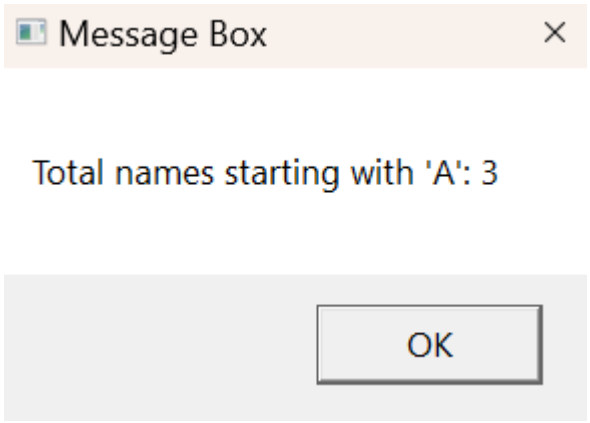
Step-by-Step Implementation

1. **Open UiPath Studio** and create a new **Process** named `Array_Name_Counter`.
2. Drag a **Sequence** container into the Designer panel and name it `Main Sequence`.
3. Go to the **Variables** panel at the bottom and create two variables:
    - `NameArray` | Variable Type: `System.String[]` (Select *Array of [T] -> String*) | Scope: `Main Sequence` | Default Value: `{"Alice", "Bob", "alex", "Charlie", "Amanda", "David"}`

    - `CountA` | Variable Type: `Int32` | Scope: `Main Sequence` | Default Value: `0`
4. Drag a **For Each** activity into the sequence.
    - Set the expression to loop through: `NameArray`.
    - In the properties panel (or activity header), ensure the argument type matches (`String`). Rename the current item variable to `name`.
5. Inside the **Body** of the For Each loop, drag an **If** activity.
    - Set the **Condition** to: `name.Trim.ToLower.StartsWith("a")`
6. In the **Then** section of the If activity, drag an **Assign** activity:
    - **To:** `CountA`
    - **Value:** `CountA + 1`
7. Leave the **Else** section empty.
8. Outside and below the *For Each* loop, drag a **Message Box** activity.
    - Set the Text to: `"Total names starting with 'A': " + CountA.ToString`
9. Click **Run File**. A message box will display `Total names starting with 'A':`


Sequence
For Each name
In *
namearr
Body
If
Condition *
name.Trim.ToLower.StartsV
Then
Assign
To variable *
Out
countA
Set value *
countA + 1
Else
Drop activity here or generate with Autopilot
Message Box
Text *
"Total names starting with

3.


Message Box
Total names starting with 'A': 3
OK

Practical 6: Excel Automation & Data Tables in UiPath

Aim

To design a UiPath application that automates reading, writing, and appending data in an Excel workbook, and demonstrates how to extract spreadsheet data into a Data Table and vice versa.

Theory

- **Excel Automation:** UiPath provides two types of Excel activities:
  - *Workbook Activities* (`UiPath.Excel.Activities`): Do not require Microsoft Excel to be installed on the machine. They execute in the background.
  - *Excel App Integration Activities*: Require MS Excel installed and run inside an *Excel Process Scope*.
- **Data Table (`System.Data.DataTable`):** A variable type that represents an in-memory database table (rows and columns) used to hold and manipulate tabular data within UiPath.
- **Key Activities:**
  - **Read Range:** Extracts data from an Excel worksheet and stores it inside a Data Table variable.
  - **Write Range:** Overwrites or creates a new sheet using data stored in a Data Table.
  - **Append Range:** Adds new rows from a Data Table to the bottom of an existing Excel sheet without deleting current data.

Step-by-Step Implementation

Step 1: Read the Desktop Excel File

1. Make sure your **Read Range** activity is sitting comfortably **inside the Do block** of your **Use Excel File** container.
2. In the **Range** input box of that *Read Range* activity, click the plus `+` icon on the right side → hover over **Excel** → select **Sheet1**.
   - *The box should now display a neat blue block reading:* `Excel.Sheet("Sheet1")`.
3. Select the *Read Range* activity card, and look at the **Properties Panel** on the right side of your screen.

4. Locate the **Save to** field under the *Output* section, click inside it, and press `Ctrl + K` on your keyboard.
5. Type `dt_Students` and press **Enter**. *(This saves your excel data into a reusable data table variable)*.

Step 2: Write the Data to a New Sheet (Data Table to Excel)

To fulfill the requirement of moving data from a data table back to Excel:

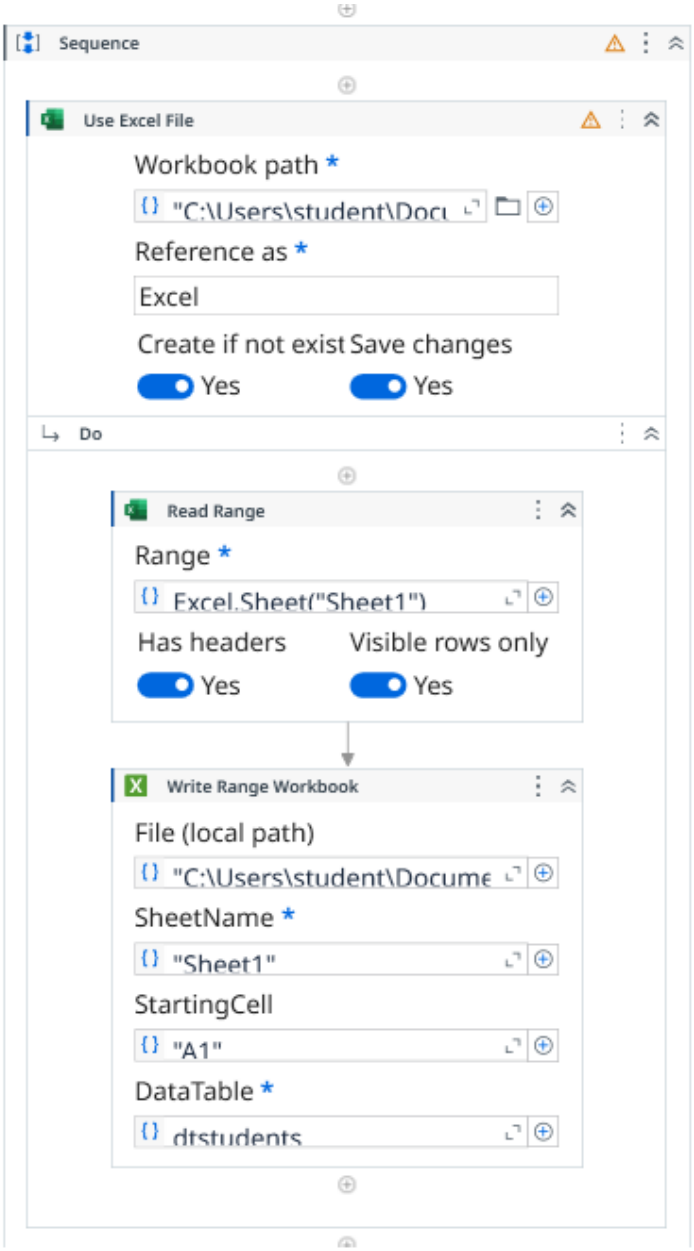
1. Go to the **Activities Panel** on the left side of your screen, search for **"Write Range"**, and drag it into your workflow.
2. Drop it **directly underneath your Read Range activity** (it must also sit *inside* the **Do block** of the *Use Excel File* card).
3. Inside the **Write Range** activity fields, set them as follows:
    - **What to write:** Click the plus `+` icon → hover over **Variables** → select your data table variable `dt_Students`.
    - **Destination:** Click the plus `+` icon → hover over **Excel** → select **Custom Input**. A text box will appear; type `"OutputSheet"` inside it (including the double quotes).
    - **Options:** Ensure that the **Exclude headers** toggle or checkbox is **turned off / unchecked** so that your column titles are copied over.

Step 3: Run Your Automated Process

1. Save your project by pressing `Ctrl + S`.
2. Look at the top left corner of UiPath Studio, click the tiny dropdown arrow next to the Play icon, and click **Run File** (or simply press `F5` on your keyboard).
3. Wait a few seconds for the execution to complete. You will see a green success message in your bottom status bar.

Step 4: Verify the Output

1. Minimize UiPath Studio and go to your Desktop.
2. Open your **StudentData.xlsx** file.
3. Look at the bottom tabs of the workbook. You will find a brand new automated sheet tab named **OutputSheet** containing a perfect copy of your data table!



	A	B	C	D
	id	Name	Department	
	1	a	it	
	2	s	ds	
	3	d	hr	
	4	f	hr	
	5	g	it	
	6	h	ds	
	7	j	it	
	8	k	ds	
	9	l	hr	

Practical 7: UI Automation and Attach Window Activity

Aim

To implement the **Attach Window** activity in UiPath to attach to an active desktop application window (such as Notepad) and perform automated UI actions (like typing text) within it.

Theory

- **Containers in UiPath:** Containers are specialized blocks used to organize automated interactions with specific applications or windows. They store a **Selector** (a collection of XML attributes that uniquely identify a window or UI element), ensuring that all activities nested inside interact with the correct application window.
- **Attach Window Activity:** A container activity used to attach to an already opened window. This is highly useful when an automation needs to shift its focus back and forth between different open applications.
- **Selector Optimization:** When you attach a window, UiPath generates a top-level selector. Any activity dropped inside it (like *Type Into* or *Click*) will inherit this selector, making the automation run faster and more reliably.

Step-by-Step Implementation

Prerequisite

Before starting your automation workflow, manually open **Notepad** on your computer.

Step-by-Step for Practical 7 (Using Modern Design):

1. Drag `Use Application/Browser` into your workflow workspace.
2. Open **Notepad** manually on your desktop so it is running in the background.
3. Inside your UiPath workflow, click the blue text link that says **"Indicate application to automate"** on the activity card.
4. Move your mouse to your open **Notepad** window and click it.
    - *UiPath will automatically link to Notepad and show a small preview snapshot inside the activity container.*

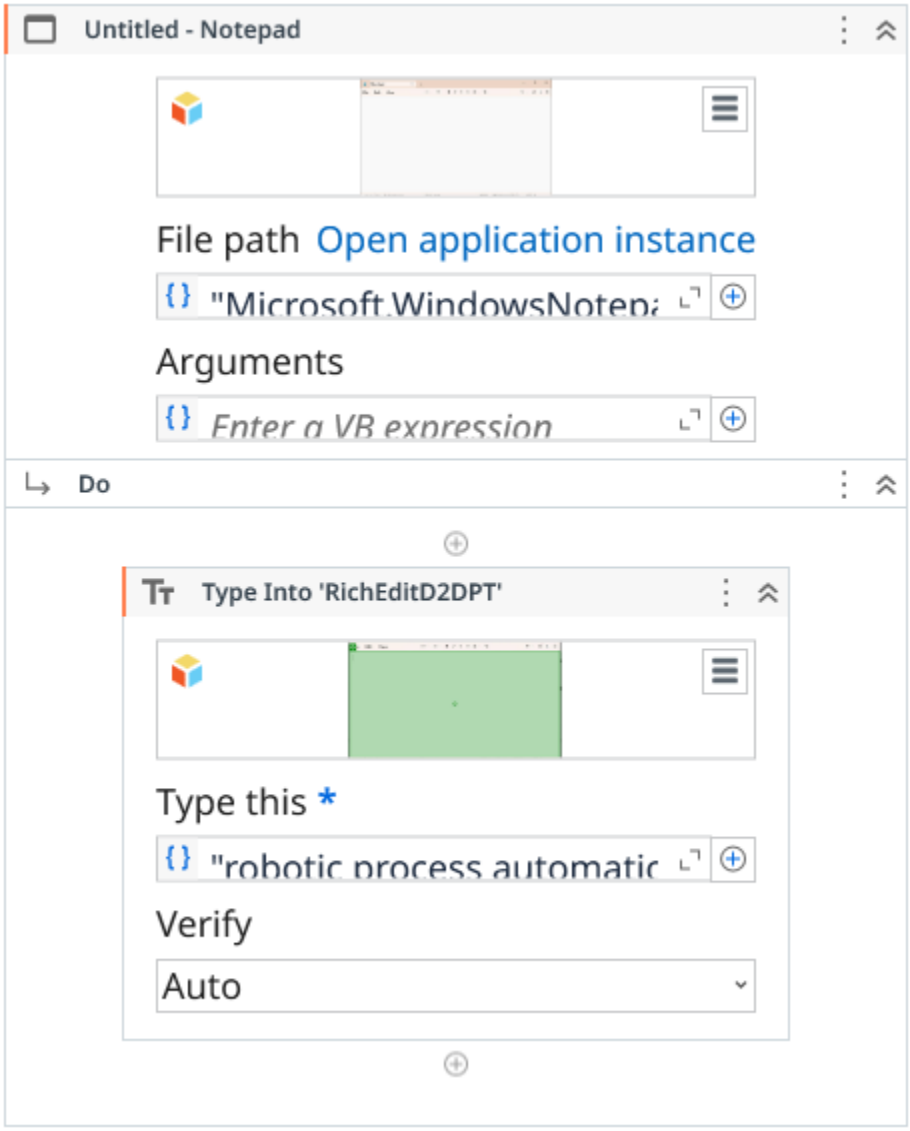
5. Inside the inner **Do** block of this activity, drag a `Type Into` activity.
6. Click **"Indicate element in window"** on that Type Into activity, and click inside the blank text workspace area of Notepad.
1. In the Text field of the Type Into activity, type your message using straight double quotes, like this:

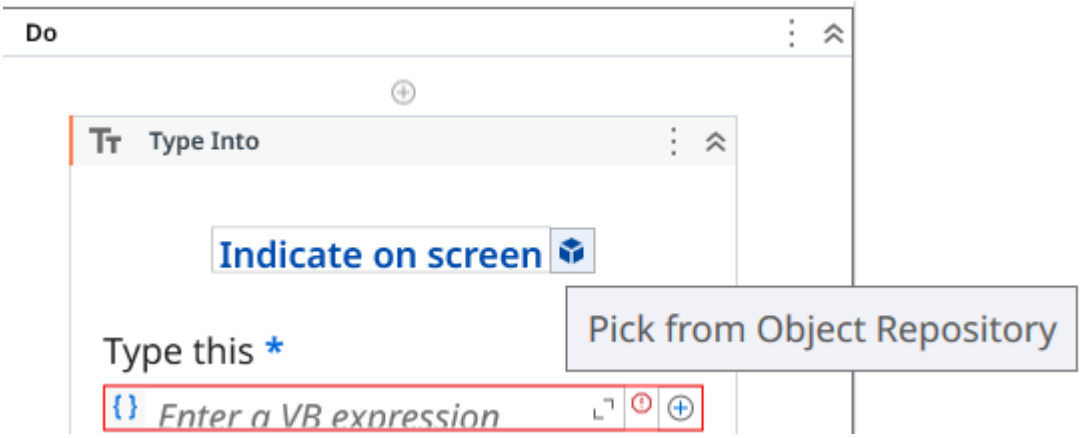
   text

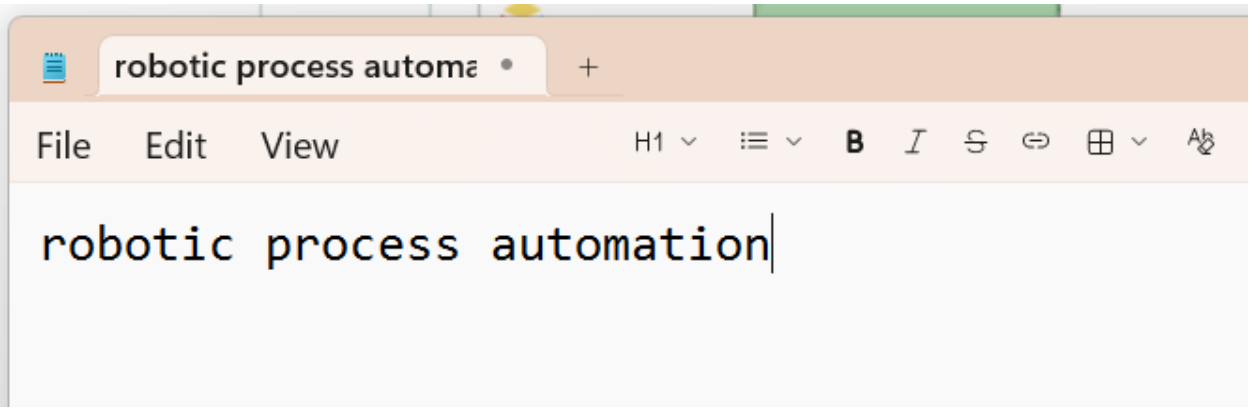
   ```
   "This text is automated using the modern window attachment tool!"
   ```
7. Use code with caution.

Once you run this file, UiPath will automatically target, attach to, and type text inside your open Notepad window!

Expected Result: UiPath will immediately find the open Notepad window, attach itself to it, pull it to the foreground, and type the designated text into the text field.


Untitled - Notepad
File path
Open application instance
"Microsoft.WindowsNotep
Arguments
Enter a VB expression
Do
Type Into 'RichEditD2DPT'
Type this *
"robotic process automatic
Verify
Auto


Do
Type Into
Indicate on screen
Pick from Object Repository
Type this *
Enter a VB expression


robotic process automa
File
Edit
View
H1
robotic process automation

Practical 7(b): Find Different Controls Using UI Explorer

Aim

To use the **UI Explorer** tool in UiPath to inspect a user interface element, analyze its structural tree properties, and validate a reliable selector for application controls.

Step-by-Step Implementation

1. **Set Up the Target Application:** Open a blank **Notepad** application window on your desktop screen and leave it open in the background.
2. **Launch UI Explorer:** In UiPath Studio, click the **Home** tab in the top-left corner to open the backstage menu. Click **Tools** in the left sidebar, then click the **UI Explorer** tile on the right to launch the standalone tool window.
3. **Indicate Element:** Click the **Indicate Element** button (mouse icon) in the top-left corner of the UI Explorer window. Your screen will freeze temporarily. Move your cursor to the open Notepad window and click inside the main white text-typing area.
4. **Explore the Visual Tree:** Look at the **Visual Tree Panel** on the left side of the UI Explorer. Expand the nodes to observe the hierarchical structural map of the Notepad application window.
5. **Modify Control Attributes:** In the **Unselected Attributes** panel (bottom-right), check the box next to `title` or `cls`. Notice how checking these parameters dynamically updates the XML code strings inside the **Selector Editor** (top-center panel).
6. **Toggle UI Frameworks:** Click the **UI Frameworks** dropdown menu on the top toolbar. Switch between **Default**, **Active Accessibility**, and **UI Automation** to see how the software structural tree mapping changes.
7. **Validate the Control:** Set the framework back to *Default* and click the **Validate** button (checkmark icon) at the top. The status bar will turn solid green, confirming that the control path is stable and verified. Close UI Explorer when finished.

Practical 7(c): Mouse Actions, Type Into, and Type Secure Text

Aim

To demonstrate basic user interface interactions in UiPath, including mouse actions (Click, Double Click, Hover), standard text typing, and entering masked secure text.

Step-by-Step Implementation

Part 1: Setting Up the Sequence & Variable

1. Open your UiPath Studio project, clear the left **Activities Search Bar**, and create a new **Sequence** inside your workflow canvas.
2. Go to the **Variables Panel** at the bottom right of the screen:
    - Click **Create Variable** and name it `SecurePass`.
    - Under **Variable Type**, select *Browse for Types*, search for `System.Security.SecureString`, and click OK.

Part 2: Adding Mouse Interactions (Click, Double Click, Hover)

3. Search for **Click** in the Activities panel. Drag the standard **Click** activity (under *UI Automation* > *Application*) into your sequence.
4. Click **"Indicate in App"** on the activity card and click the **File** menu option at the top left of your open Notepad.
5. Drag a **second Click** activity right underneath the first one. Select it, look at the **Properties Panel** on the right side of the screen, find **Click Type**, and change it from *Single* to `Double`.
6. Drag a **third Click** activity into the sequence. Go to its properties panel, find the **Action Type** (or Click Type) dropdown, and change it to `Hover` (or use a standalone *Hover* activity if available).

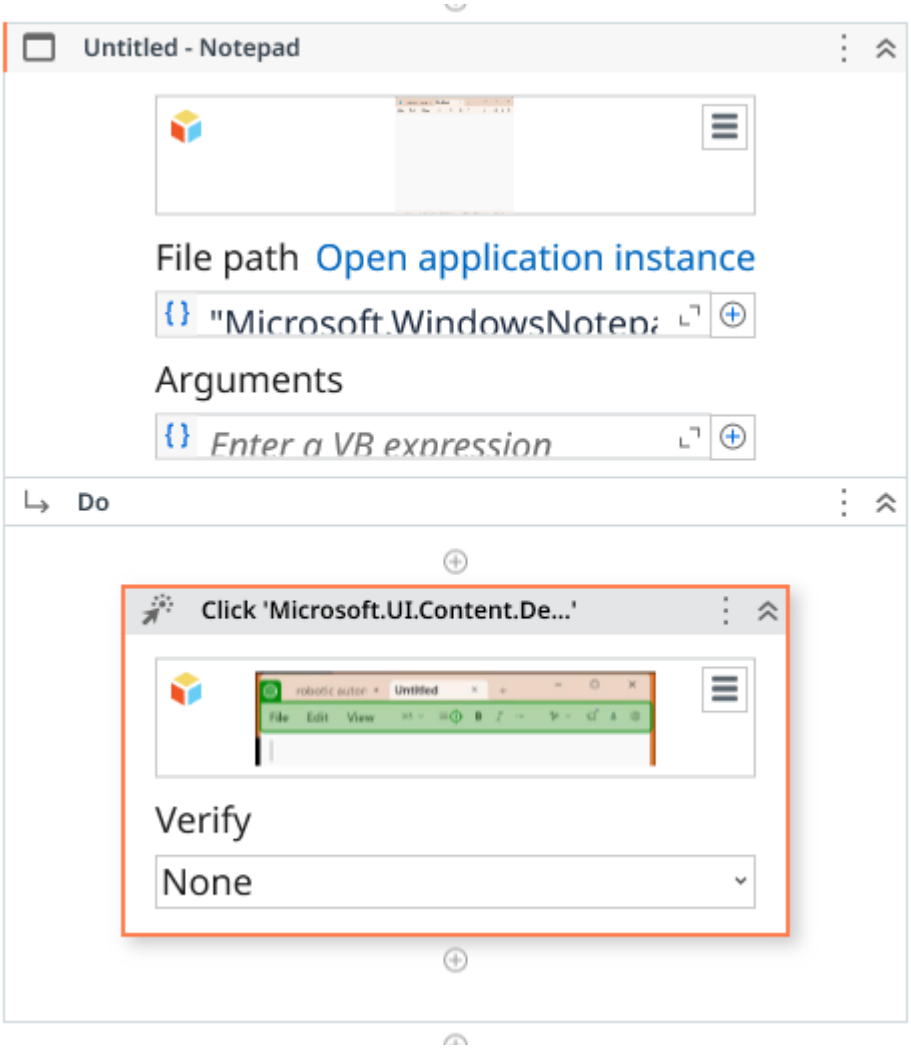
Part 3: Adding Text Inputs (Type Into & Type Secure Text)

7. Search for **Type Into** in the Activities panel and drag it right underneath your click activities.

8. Click **"Indicate in App"** and click inside the main white text area of your open Notepad. In its text box input field, type exactly: `"UiPath standard typing text."` (include the double quotes).
9. Search for **Type Secure Text** in the activities panel and drag it to the bottom of your sequence stack.
10. Click **"Indicate in App"** and select the same Notepad white typing area.
11. Inside the **Secure text** input field on the activity card (or in its right Properties panel), type your variable name: `SecurePass`.

Part 4: Run the Process

12. Make sure your target Notepad window is visible on your screen, click the dropdown arrow next to the Play icon at the top left of Studio, and click **Run File** (or press `F5`). The robot will sequentially click, double-click, hover, type standard text, and type masked secure text into Notepad.



Practical 8(a): Element, Image, and System Triggering Events

Aim

To implement a workflow that listens for and responds to physical computer peripheral triggers, visual layout changes, or UI state mutations using a **Trigger Scope**.

Theory

- **Trigger Scope:** A structural action container that runs concurrently in the background. It continuously monitors your active operating system interface for specific specified rules before firing an execution event loop.
- **Key Press Trigger (System):** Activates immediately when a designated keyboard button or shortcut hotkey is hit.
- **Element State Change Trigger (UI Element):** Automatically executes when a target digital element box on an application screen changes state (e.g., Appears or Disappears).
- **Image Trigger (Visual):** Continuously scans screen matrices until a specific cropped visual asset or logo graphic matches what is being displayed live.

Step-by-Step Implementation

1. Open **Notepad** on your screen.
2. In UiPath Studio, clear your activities search box, search for **Trigger Scope**, and drag it into a new sequence canvas.
3. **Add System Trigger:** Search for **Key Press Trigger** in the activities panel. Drag it and drop it directly into the inner **Triggers** section box of the *Trigger Scope* card.
    - Click inside its properties on the right, and change the **Key** field by choosing `"enter"` or typing `"esc"` in quotes.
4. **Add Element Trigger:** Search for **Element State Change Trigger** and drag it right below the key trigger.

    - Click *"Indicate on screen"* and click on the Notepad layout window. Set the state property to `Disappear`.
5. **Add Image Trigger:** Search for **Image Trigger** and drag it into the trigger box space. Click *"Indicate on screen"* and drag a crop box around a static background shortcut logo file icon. Set it to trigger when it `Appears`.
6. **Configure the Action Loop:** Drag a standard **Message Box** activity and drop it inside the lower **Actions** section block of your *Trigger Scope*.
    - Set its text string box property to: `"An automation event trigger was captured successfully!"`
7. Click **Run File (`F5`)**. Click your designated key on your keyboard or close your background notepad instance, and see the trigger instantly throw the automated alert popup window!

Practical 8(b): Screen Scraping Methods (Full Text, Native, OCR)

Aim

To extract text information from different application surfaces and digital documents using the three core scraping methods in UiPath.

Theory

- **Full Text Scraping:** The default method in UiPath. It is fast, has 100% digital extraction precision, and can read hidden background items, but cannot capture structural fonts or text positions.
- **Native Scraping:** Pulls live textual characters along with layout attributes like text screen positions, sizing metrics, and font decorations.
- **OCR Scraping (Optical Character Recognition):** Visually processes graphic layers to extract characters out of image files, remote desktop streams, or unreadable scanned paper PDFs.

Step-by-Step Implementation

1. Open any text file or web page window containing a clean written description in the background.
2. Go to the top horizontal ribbon toolbar dashboard of UiPath Studio and click directly on the **Screen Scraping** wizard icon button.
3. Move your cursor over your target background document template and click once. A dedicated **Screen Scraper Wizard window** overlay will automatically open.
4. Locate the **Scraping Method** option dropdown selector selection panel inside the wizard workspace:
    - Select **Full Text** to view standard text extractions.
    - Switch it to **Native** to see text position coordinates map out.
    - Toggle it to **OCR** to choose an imaging model engine like *Tesseract OCR* or *UiPath Screen OCR* for visual scans.

5. Click the **Finish** button at the bottom right corner of the wizard panel.
6. UiPath will automatically generate an error-free, formatted sequence extraction stack right on your active canvas workspace containing your harvested data strings pre-saved to a functional text variable!

Practical 8(c): Automation with Plug-ins (Excel, Word, PDF, Mail)

Aim

To install necessary library packages and automate structured information exchanges using external application extension modules (Excel, Word, PDF, Mail).

Step-by-Step Implementation

1. **Install Plug-ins:** Click on **Manage Packages** at the top ribbon menu. Select **All Packages**, search for `UiPath.PDF.Activities`, `UiPath.Word.Activities`, and `UiPath.Mail.Activities`, and click *Install* > *Save*.
2. **Word & Excel Setup:** Drag a **Word Application Scope** activity container card into your process sequence. Drop a **Read Text** activity inside it to read a `.docx` text document. Directly below it, drop your working **Use Excel File** block to copy that data out into a spreadsheet cell.
3. **PDF Extraction:** Clear your activity library search bar, search for **Read PDF Text**, and drag it onto your sequence panel stack.
    - Set the **File path** property string field to look at a local file: `"C:\Users\student\Desktop\Document.pdf"`.
    - Click inside its **Text** output field properties block on the right, press **`Ctrl + K`**, and name your output text variable **`pdfText`**.
4. **Mail Delivery:** Search for **Send SMTP Mail Message** (or *Send Outlook Mail Message*) and drop it at the very bottom of your execution layout stack.
    - Fill the **To** input field address with your receiver string: `"test@gmail.com"`.
    - Write your topic inside the **Subject** input box: `"Automated Data Report Upload"`.
    - Pass your processed extraction variable name into the **Body** field box: **`pdfText`** (No quotes for variable entries!). Run the file to execute the integrated pipeline

Practical 9(a): Automate the Process of Send Mail Event

Aim

To configure an automation workflow that programmatically triggers and delivers transactional email messages.

Theory

- **Mail Activities Plugin:** Allows corporate systems to distribute digital notices, attachment data files, or system failure warnings directly to remote email services using mail protocols like SMTP or Outlook APIs.

Step-by-Step Implementation

1. Search for **"Send SMTP Mail Message"** in your left-hand Activities Panel and drag it onto a blank editing canvas sequence.
2. In the Properties Panel on the right side of the screen, expand the **Host** configuration menu:
    - **Port:** `587`
    - **Server:** `"://gmail.com"` (with double quotes)
3. Expand the **Logon** properties section block grid:
    - **Email:** `"your_account@gmail.com"`
    - **Password:** Use your account string or pass your secure system string credentials variable.
4. Fill out the **Receiver & Text inputs** directly on the card interface:
    - **To:** `"recipient_address@gmail.com"`
    - **Subject:** `"UiPath Automated Studio Notification Alert"`
    - **Body:** `"Hello, this email was dispatched autonomously by a UiPath software application bot execution stream."`

5. Press `F5` to run and monitor your incoming mailbox inbox layout tabs to confirm delivery.

Practical 9(b): Launching an Assistant Bot on a Keyboard Event

Aim

To launch a secondary helper application loop triggered via a physical keyboard key sequence command rule.

Step-by-Step Implementation

1. Drag a fresh **Trigger Scope** activity card container into a new workflow sequence block.
2. Search for **Key Press Trigger** inside your library and drop it directly into the inner **Triggers** slot box of that scope card.
3. Look at its properties panel layout configurations on the right:
    - In the **Key** field, type out your activation hotkey string inside double quotes: `"s"`
    - In the **KeyModifiers** dropdown option array, check the box next to `Alt` (or *Ctrl*).
4. Inside the lower **Actions** section block area of your *Trigger Scope*, drag a **Use Application/Browser** container activity.
5. Click *"Indicate application"* and select your open **UiPath Assistant** program client desktop interface (or indicate any terminal dashboard assistant dashboard pane tool).
6. Click **Run File**. Now, pressing `Alt + S` on your laptop keyboard will instantly call up and open the automated assistant dashboard frame window!

Practical 9(c): Exception Handling in UiPath (Try Catch)

Aim

To implement a robust error management pipeline using **Try Catch** and **Throw** modules to prevent workflow crashes during execution runtime blocks.

Theory

- **Try Block:** Houses the core functional automation steps that could potentially encounter an unpredictable error (like a missing file path string or a network failure).
- **Catch Block:** Captures specific system faults if they happen inside the *Try* block and safely resolves them without terminating the software robot execution completely.
- **Finally Block:** A final sequence container box that is guaranteed to run at the absolute end, regardless of whether a runtime failure occurred or skipped.

Step-by-Step Implementation

1. Drag a **Try Catch** activity into your primary sequence canvas workspace.
2. Click inside the **Try** block tab space area, and drag an intentional error module inside it (e.g., drop a *Read Range Workbook* activity pointing to a completely blank, non-existent file path: `"C:\MissingFile.xlsx"`).
3. Click on the **Catches** section line tab area located just below your Try block box. Click the **"Add new catch"** dropdown menu and select: `System.Exception`.
4. Inside the inner body canvas sequence box of this newly created *Catch* block layout, drag a **Message Box** activity.
5. Set its text property string to output the native error log message safely to the user:

"An error occurred but was caught safely. Details: " + exception.Message

6.

7. Click inside the **Finally** section tab below. Drop a **Log Message** or a message box saying `"Cleanup complete. System resetting."`
8. Click **Run File**. Instead of crashing with a red warning alert message panel, your robot will capture the missing file issue smoothly and display your custom message box safely!

Practical 9(d): Use of Config Files (.xlsx) in UiPath

Aim

To isolate runtime variables, asset keys, and file path parameters away from hardcoded automation structures into an external **Config.xlsx** spreadsheet profile file.

Theory

- **Configuration Files:** Key-Value pair tables that allow administrators to adjust paths or email targets in production without ever opening or modifying the source code inside UiPath Studio.

Step-by-Step Implementation

1. Create an Excel sheet on your desktop named `Config.xlsx`. Format your sheets with columns labeled **`Name`** and **`Value`**:
    - Row 1: `FilePath` | `"C:\Users\student\Desktop\StudentData.xlsx"`
    - Row 2: `SupportEmail` | `"admin@company.com"`
2. Inside UiPath Studio, drop a **Use Excel File** container and a **Read Range** activity inside your initialization block sequence. Read your `Config.xlsx` file and output it to a data table variable named `dt_Config`.
3. Drag an **Assign** activity below your configuration reader card loop to parse the table.
4. Set the **To** property box to hold your process path variable: `TargetFileString`.
5. Set the target extraction dictionary value lookup string expression in the **Value** property input field box using a quick data table lookup query filter string line:

dt_Config.AsEnumerable().Where(Function(r) r("Name").ToString = "FilePath").Select(Function(r) r("Value").ToString).FirstOrDefault()

6. Pass your decoupled variable `TargetFileString` directly into your main operational process inputs instead of a hardcoded path string. This allows for dynamic adjustments without editing the source code.